

GLOBAL  
EDITION



# C How to Program

with an introduction to C++ {

EIGHTH EDITION

Paul Deitel • Harvey Deitel



ALWAYS LEARNING

PEARSON

# HOW TO

# PROG

# RAM<sub>EIG</sub>

# HTH EDITION

**GLOBAL EDITION**

**with an introduction to C++**

*This page intentionally left blank*

**HOW TO**

**PROG**

**RAM<sub>EIG</sub>**

**HTH EDITION**

**GLOBAL EDITION**

**with an introduction to C++**

**Paul Deitel**

*Deitel & Associates, Inc.*

**Harvey Deitel**

*Deitel & Associates, Inc.*

**Global Edition contributions by**

**Piyali Sengupta**



Boston Columbus Hoboken Indianapolis New York San Francisco  
Amsterdam Cape Town Dubai London Madrid Milan Munich Paris Montreal  
Toronto Delhi Mexico City Sao Paulo Sydney Hong Kong Seoul Singapore Taipei  
Tokyo ~

Vice President and Editorial Director, ECS: Marcia J. Horton  
Executive Editor: Tracy Johnson (Dunkelberger)  
Editorial Assistant: Kelsey Loanes  
Assistant Acquisitions Editor, Global Editions: Aditee Agarwal  
Program Manager: Carole Snyder  
Project Manager: Robert Engelhardt  
Project Editor, Global Editions: K.K. Neelakantan  
Media Team Lead: Steve Wright  
R&P Manager: Rachel Youdelman  
R&P Senior Project Manager: William Opaluch  
Senior Operations Specialist: Maura Zaldivar-Garcia  
Senior Manufacturing Controller, Global Editions: Kay Holman  
Inventory Manager: Bruce Boundy  
Marketing Manager: Demetrius Hall  
Product Marketing Manager: Bram Van Kempen  
Media Production Manager, Global Editions: Vikram Kumar  
Marketing Assistant: Jon Bryant  
Cover Designer: Chuti Prasertsith / Michael Rutkowski / Marta Samsel  
Cover Art: © Igoror / Shutterstock

Pearson Education Limited  
Edinburgh Gate  
Harlow  
Essex CM20 2JE  
England

and Associated Companies throughout the world

Visit us on the World Wide Web at:  
[www.pearsonglobaleditions.com](http://www.pearsonglobaleditions.com)

© Pearson Education Limited 2016

The rights of Paul Deitel and Harvey Deitel to be identified as the authors of this work have been asserted by them in accordance with the Copyright, Designs and Patents Act 1988.

*Authorized adaptation from the United States edition, entitled C How to Program: with an introduction to C++, 8<sup>th</sup> edition, ISBN 978-0-13-397689-2, by Paul Deitel and Harvey Deitel published by Pearson Education © 2016.*

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without either the prior written permission of the publisher or a license permitting restricted copying in the United Kingdom issued by the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS.

All trademarks used herein are the property of their respective owners. The use of any trademark in this text does not vest in the author or publisher any trademark ownership rights in such trademarks, nor does the use of such trademarks imply any affiliation with or endorsement of this book by such owners.

British Library Cataloguing-in-Publication Data

A catalogue record for this book is available from the British Library

10 9 8 7 6 5 4 3 2 1

ISBN 10: 1-292-11097-X  
ISBN 13: 978-1-292-11097-4

Typeset by GEX Publishing Services

Printed in Malaysia

*In memory of Dennis Ritchie,*

# *creator of the C programming language and co-creator of the UNIX operating system.*

*Paul and Harvey Deitel*

## Trademarks

DEITEL, the double-thumbs-up bug and DIVE INTO are registered trademarks of Deitel and Associates, Inc.

Apple, Xcode, Swift, Objective-C, iOS and OS X are trademarks or registered trademarks of Apple, Inc. Java is a registered trademark of Oracle and/or its affiliates.

Microsoft and/or its respective suppliers make no representations about the suitability of the information contained in the documents and related graphics published as part of the services for any purpose. All such documents and related graphics are provided “as is” without warranty of any kind. Microsoft and/or its respective suppliers hereby disclaim all warranties and conditions with regard to this information, including all warranties and conditions of merchantability, whether express, implied or statutory, fitness for a particular purpose, title and non-infringement. In no event shall Microsoft and/or its respective suppliers be liable for any special, indirect or consequential damages or any damages whatsoever resulting from loss of use, data or profits, whether in an action of contract, negligence or other tortious action, arising out of or in connection with the use or performance of information available from the services.

The documents and related graphics contained herein could include technical inaccuracies or typographical errors. Changes are periodically added to the information herein. Microsoft and/or its respective suppliers may make improvements and/or changes in the product(s) and/or the program(s) described herein at any time. Partial screen shots may be viewed in full within the software version specified.

Other names may be trademarks of their respective owners.

**Appendices F, G and H are PDF documents posted online at the book's Companion Website (located at [www.pearsonglobaleditions.com/deitel](http://www.pearsonglobaleditions.com/deitel)).**

## Preface 23

### **1 Introduction to Computers, the Internet and the Web 33**

1.1 Introduction 34 1.2 Hardware and Software 35 1.2.1 Moore's Law 35 1.2.2 Computer Organization 36 1.3 Data Hierarchy 37 1.4 Machine Languages, Assembly Languages and High-Level Languages 40 1.5 The C Programming Language 41 1.6 C Standard Library 42 1.7 C++ and Other C-Based Languages 43 1.8 Object Technology 44 1.8.1 The Automobile as an Object 45 1.8.2 Methods and Classes 45 1.8.3 Instantiation 45 1.8.4 Reuse 45 1.8.5 Messages and Method Calls 46 1.8.6 Attributes and Instance Variables 46 1.8.7 Encapsulation and Information

|                                                                  |    |                                                                  |    |                                                                |    |
|------------------------------------------------------------------|----|------------------------------------------------------------------|----|----------------------------------------------------------------|----|
| Hiding                                                           | 46 | 1.8.8 Inheritance                                                | 46 | 1.9 Typical C Program—Development Environment                  | 47 |
| 1.9.1 Phase 1: Creating a Program                                | 48 | 1.9.2 Phases 2 and 3: Preprocessing and Compiling a C Program    | 48 | 1.9.3 Phase 4: Linking                                         | 48 |
| 1.9.4 Phase 5: Loading                                           | 49 | 1.9.5 Phase 6: Execution                                         | 49 | 1.9.6 Problems That May Occur at Execution Time                | 49 |
| 1.9.7 Standard Input, Standard Output and Standard Error Streams | 49 | 1.10 Test-Driving a C Application in Windows, Linux and Mac OS X | 49 | 1.10.1 Running a C Application from the Windows Command Prompt | 50 |
| 1.10.2 Running a C Application Using GNU C with Linux            | 53 |                                                                  |    |                                                                |    |

## 8 Contents

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| 1.10.3 Running a C Application Using the Terminal on Mac OS X                   | 56 |
| 1.11 Operating Systems                                                          | 59 |
| 1.11.1 Windows—A Proprietary Operating System                                   | 59 |
| 1.11.2 Linux—An Open-Source Operating System                                    | 59 |
| 1.11.3 Apple's Mac OS X; Apple's iOS for iPhone®, iPad® and iPod Touch® Devices | 60 |
| 1.11.4 Google's Android                                                         | 60 |
| 1.12 The Internet and World Wide Web                                            | 61 |
| 1.12.1 The Internet: A Network of Networks                                      | 61 |
| 1.12.2 The World Wide Web: Making the Internet User-Friendly                    | 61 |
| 1.12.3 Web Services                                                             | 62 |
| 1.12.4 Ajax                                                                     | 64 |
| 1.12.5 The Internet of Things                                                   | 64 |
| 1.13 Some Key Software Terminology                                              | 64 |
| 1.14 Keeping Up-to-Date with Information Technologies                           | 66 |

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>2 Introduction to C Programming</b>                 | <b>71</b> |
| 2.1 Introduction                                       | 72        |
| 2.2 A Simple C Program: Printing a Line of Text        | 72        |
| 2.3 Another Simple C Program: Adding Two Integers      | 76        |
| 2.4 Memory Concepts                                    | 80        |
| 2.5 Arithmetic in C                                    | 81        |
| 2.6 Decision Making: Equality and Relational Operators | 85        |
| 2.7 Secure C Programming                               | 89        |

|                                                                                                           |            |
|-----------------------------------------------------------------------------------------------------------|------------|
| <b>3 Structured Program Development in C</b>                                                              | <b>101</b> |
| 3.1 Introduction                                                                                          | 102        |
| 3.2 Algorithms                                                                                            | 102        |
| 3.3 Pseudocode                                                                                            | 102        |
| 3.4 Control Structures                                                                                    | 103        |
| 3.5 The if Selection Statement                                                                            | 105        |
| 3.6 The if...else Selection Statement                                                                     | 106        |
| 3.7 The while Iteration Statement                                                                         | 110        |
| 3.8 Formulating Algorithms Case Study 1: Counter-Controlled Iteration                                     | 111        |
| 3.9 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 2: Sentinel-Controlled Iteration | 114        |
| 3.10 Formulating Algorithms with Top-Down, Stepwise Refinement Case Study 3: Nested Control Statements    | 120        |
| 3.11 Assignment Operators                                                                                 | 124        |
| 3.12 Increment and Decrement Operators                                                                    | 125        |
| 3.13 Secure C Programming                                                                                 | 127        |

|                            |            |
|----------------------------|------------|
| <b>4 C Program Control</b> | <b>145</b> |
| 4.1 Introduction           | 146        |

Contents 9

|                                                 |     |
|-------------------------------------------------|-----|
| 4.2 Iteration Essentials                        | 146 |
| 4.3 Counter-Controlled Iteration                | 147 |
| 4.4 for Iteration Statement                     | 148 |
| 4.5 for Statement: Notes and Observations       | 151 |
| 4.6 Examples Using the for Statement            | 152 |
| 4.7 switch Multiple-Selection Statement         | 155 |
| 4.8 do...while Iteration Statement              | 161 |
| 4.9 break and continue Statements               | 162 |
| 4.10 Logical Operators                          | 164 |
| 4.11 Confusing Equality (==) and Assignment (=) |     |

|                                   |                                                      |                                            |                                              |                    |                                                     |
|-----------------------------------|------------------------------------------------------|--------------------------------------------|----------------------------------------------|--------------------|-----------------------------------------------------|
| <b>5 C Functions 189</b>          | 5.1 Introduction 190                                 | 5.2 Modularizing Programs in C 190         | 5.3 Math Library Functions 191               | 5.4 Functions 193  | 5.5 Function Definitions 193                        |
| 5.5.1 square Function 194         | 5.5.2 maximum Function 197                           | 5.6 Function Prototypes: A Deeper Look 198 | 5.7 Function Call Stack and Stack Frames 200 | 5.8 Headers 204    | 5.9 Passing Arguments By Value and By Reference 205 |
| 5.10 Random Number Generation 206 | 5.11 Example: A Game of Chance; Introducing enum 210 | 5.12 Storage Classes 214                   | 5.13 Scope Rules 216                         | 5.14 Recursion 219 | 5.15 Example Using Recursion: Fibonacci Series 222  |
| 5.16 Recursion vs. Iteration 226  | 5.17 Secure C Programming 227                        |                                            |                                              |                    |                                                     |

|                                                                                |                                                                          |                                                                                                                 |                         |                        |
|--------------------------------------------------------------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|-------------------------|------------------------|
| <b>6 C Arrays 246</b>                                                          | 6.1 Introduction 247                                                     | 6.2 Arrays 247                                                                                                  | 6.3 Defining Arrays 249 | 6.4 Array Examples 249 |
| 6.4.1 Defining an Array and Using a Loop to Set the Array's Element Values 249 | 6.4.2 Initializing an Array in a Definition with an Initializer List 250 | 6.4.3 Specifying an Array's Size with a Symbolic Constant and Initializing Array Elements with Calculations 252 |                         |                        |

## 10 Contents

|                                                        |                                                                                 |                                                             |                                                                                  |                                                                  |
|--------------------------------------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------|
| 6.4.4 Summing the Elements of an Array 253             | 6.4.5 Using Arrays to Summarize Survey Results 254                              | 6.4.6 Graphing Array Element Values with Histograms 256     | 6.4.7 Rolling a Die 60,000,000 Times and Summarizing the Results in an Array 257 | 6.5 Using Character Arrays to Store and Manipulate Strings 257   |
| 6.5.1 Initializing a Character Array with a String 258 | 6.5.2 Initializing a Character Array with an Initializer List of Characters 258 | 6.5.3 Accessing the Characters in a String 258              | 6.5.4 Inputting into a Character Array 258                                       | 6.5.5 Outputting a Character Array That Represents a String 259  |
| 6.5.6 Demonstrating Character Arrays 259               | 6.6 Static Local Arrays and Automatic Local Arrays 260                          | 6.7 Passing Arrays to Functions 262                         | 6.8 Sorting Arrays 266                                                           | 6.9 Case Study: Computing Mean, Median and Mode Using Arrays 268 |
| 6.10 Searching Arrays 273                              | 6.10.1 Searching an Array with Linear Search 273                                | 6.10.2 Searching an Array with Binary Search 274            | 6.11 Multidimensional Arrays 278                                                 | 6.11.1 Illustrating a Double-Subscripted Array 278               |
| 6.11.2 Initializing a Double-Subscripted Array 279     | 6.11.3 Setting the Elements in One Row 281                                      | 6.11.4 Totaling the Elements in a Two-Dimensional Array 281 | 6.11.5 Two-Dimensional Array Manipulations 282                                   | 6.12 Variable-Length Arrays 285                                  |
| 6.13 Secure C Programming 288                          |                                                                                 |                                                             |                                                                                  |                                                                  |

|                                                                        |                      |                                                         |                           |                                                     |                                                 |
|------------------------------------------------------------------------|----------------------|---------------------------------------------------------|---------------------------|-----------------------------------------------------|-------------------------------------------------|
| <b>7 C Pointers 306</b>                                                | 7.1 Introduction 307 | 7.2 Pointer Variable Definitions and Initialization 308 | 7.3 Pointer Operators 309 | 7.4 Passing Arguments to Functions by Reference 311 | 7.5 Using the const Qualifier with Pointers 315 |
| 7.5.1 Converting a String to Uppercase Using a Non-Constant Pointer to |                      |                                                         |                           |                                                     |                                                 |

|                                                    |                                                 |
|----------------------------------------------------|-------------------------------------------------|
| Non-Constant Data 316                              | 7.5.2 Printing a String One Character at a Time |
| Using a Non-Constant Pointer to Constant Data 317  | 7.5.3 Attempting to                             |
| Modify a Constant Pointer to Non-Constant Data 319 | 7.5.4 Attempting to                             |
| Modify a Constant Pointer to Constant Data 320     | 7.6 Bubble Sort Using                           |
| Pass-by-Reference 321                              | 7.7 sizeof Operator 324                         |
| 7.8 Pointer Expressions and                        | Pointer Arithmetic 327                          |
| 7.8.1 Allowed Operators for Pointer Arithmetic 327 |                                                 |
| 7.8.2 Aiming a Pointer at an Array 327             |                                                 |
|                                                    | <a href="#">Contents</a> 11                     |

|                                                     |                                                |
|-----------------------------------------------------|------------------------------------------------|
| 7.8.3 Adding an Integer to a Pointer 328            | 7.8.4 Subtracting an Integer                   |
| from a Pointer 328                                  | 7.8.5 Incrementing and Decrementing a Pointer  |
| 328                                                 | 7.8.6 Subtracting One Pointer from Another 329 |
| 7.8.7 Assigning                                     | Pointers to One Another 329                    |
| 7.8.8 Pointer to void 329                           | 7.8.9 Comparing                                |
| Pointers 329                                        |                                                |
| 7.9 Relationship between Pointers and Arrays 330    | 7.9.1 Pointer/Offset                           |
| Notation 330                                        | 7.9.2 Pointer/Index Notation 331               |
| 7.9.3 Cannot Modify an                              | Array Name with Pointer Arithmetic 331         |
| 7.9.4 Demonstrating Pointer                         | Indexing and Offsets 331                       |
| 7.9.5 String Copying with Arrays and                | Pointers 332                                   |
| 7.10 Arrays of Pointers 334                         | 7.11 Case Study: Card Shuffling and Dealing    |
| Simulation 335                                      | 7.12 Pointers to Functions 340                 |
| 7.12.1 Sorting in Ascending or Descending Order 340 | 7.12.2 Using Function                          |
| Pointers to Create a Menu-Driven System 343         | 7.13 Secure C Programming                      |
|                                                     | 345                                            |

## 8 C Characters and Strings 365

|                                            |                                |
|--------------------------------------------|--------------------------------|
| 8.1 Introduction 366                       | 8.2                            |
| Fundamentals of Strings and Characters 366 | 8.3 Character-Handling Library |
| 368                                        |                                |

|                                                                      |                                            |
|----------------------------------------------------------------------|--------------------------------------------|
| 8.3.1 Functions isdigit, isalpha, isalnum and isxdigit 368           | 8.3.2 Functions                            |
| islower, isupper, tolower and toupper 371                            | 8.3.3 Functions isspace, iscntrl, ispunct, |
| isprint and isgraph 372                                              |                                            |
| 8.4 String-Conversion Functions 374                                  | 8.4.1 Function strtod 374                  |
| 8.4.2 Function                                                       | strtoul 375                                |
| 8.4.3 Function strtoul 376                                           |                                            |
| 8.5 Standard Input/Output Library Functions 376                      | 8.5.1 Functions fgetc and                  |
| putchar 377                                                          | 8.5.2 Function getchar 378                 |
| 8.5.3 Function sprintf 379                                           | 8.5.4                                      |
| Function sscanf 380                                                  |                                            |
| 8.6 String-Manipulation Functions of the String-Handling Library 381 | 8.6.1                                      |
| Functions strcpy and strncpy 382                                     | 8.6.2 Functions strcat and strncat 382     |
| 8.7 Comparison Functions of the String-Handling Library 383          | 8.8 Search                                 |
| Functions of the String-Handling Library 385                         | 8.8.1 Function strchr 386                  |
| 8.8.2                                                                | Function strcspn 387                       |
| 8.8.3 Function strpbrk 387                                           | 8.8.4 Function strrchr 388                 |
| 12 <a href="#">Contents</a>                                          |                                            |

|                                                         |                            |
|---------------------------------------------------------|----------------------------|
| 8.8.5 Function strspn 389                               | 8.8.6 Function strstr 389  |
| 8.8.7 Function strtok                                   | 390                        |
| 8.9 Memory Functions of the String-Handling Library 391 | 8.9.1 Function             |
| memcpy 392                                              | 8.9.2 Function memmove 393 |
| 8.9.3 Function memcmp 394                               |                            |
| 8.9.4 Function memchr 394                               | 8.9.5 Function memset 395  |
| 8.10 Other Functions of the String-Handling Library 395 | 8.10.1 Function            |

|                                       |                                                                              |                                                                                  |                                                              |                                                         |                                                      |
|---------------------------------------|------------------------------------------------------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------|---------------------------------------------------------|------------------------------------------------------|
| <b>9 C Formatted Input/Output 409</b> | 9.1 Introduction 410                                                         | 9.2 Streams 410                                                                  | 9.3 Formatting Output with printf 410                        | 9.4 Printing Integers 411                               | 9.5 Printing Floating-Point Numbers 412              |
|                                       | 9.5.1 Conversion Specifiers e, E and f 413                                   | 9.5.2 Conversion Specifiers g and G 413                                          | 9.5.3 Demonstrating Floating-Point Conversion Specifiers 414 |                                                         |                                                      |
|                                       | 9.6 Printing Strings and Characters 414                                      | 9.7 Other Conversion Specifiers 415                                              | 9.8 Printing with Field Widths and Precision 416             |                                                         |                                                      |
|                                       | 9.8.1 Specifying Field Widths for Printing Integers 416                      | 9.8.2 Specifying Precisions for Integers, Floating-Point Numbers and Strings 417 | 9.8.3 Combining Field Widths and Precisions 418              | 9.9 Using Flags in the printf Format Control String 419 | 9.9.1 Right and Left Justification 419               |
|                                       | 9.9.2 Printing Positive and Negative Numbers with and without the + Flag 420 | 9.9.3 Using the Space Flag 420                                                   | 9.9.4 Using the # Flag 421                                   | 9.9.5 Using the o Flag 421                              | 9.10 Printing Literals and Escape Sequences 422      |
|                                       | 9.11 Reading Formatted Input with scanf 422                                  | 9.11.1 scanf Syntax 423                                                          | 9.11.2 scanf Conversion Specifiers 423                       | 9.11.3 Reading Integers with scanf 424                  | 9.11.4 Reading Floating-Point Numbers with scanf 425 |
|                                       | 9.11.5 Reading Characters and Strings with scanf 425                         | 9.11.6 Using Scan Sets with scanf 426                                            |                                                              |                                                         |                                                      |

[Contents 13](#)

|                                          |                                                   |                               |
|------------------------------------------|---------------------------------------------------|-------------------------------|
| 9.11.7 Using Field Widths with scanf 427 | 9.11.8 Skipping Characters in an Input Stream 428 | 9.12 Secure C Programming 429 |
|------------------------------------------|---------------------------------------------------|-------------------------------|

|                                                                       |                       |                                |                                        |                                                  |                                |                                                           |                                  |                                                    |                                          |                  |                                                                          |                 |                               |                                                       |                                                |                                 |                            |                                                   |                                                                  |                                                                                       |                                                              |                                         |                      |                                 |                                                                         |                                |                                 |                                           |                                |
|-----------------------------------------------------------------------|-----------------------|--------------------------------|----------------------------------------|--------------------------------------------------|--------------------------------|-----------------------------------------------------------|----------------------------------|----------------------------------------------------|------------------------------------------|------------------|--------------------------------------------------------------------------|-----------------|-------------------------------|-------------------------------------------------------|------------------------------------------------|---------------------------------|----------------------------|---------------------------------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------|-----------------------------------------|----------------------|---------------------------------|-------------------------------------------------------------------------|--------------------------------|---------------------------------|-------------------------------------------|--------------------------------|
| <b>10 C Structures, Unions, Bit Manipulation and Enumerations 436</b> | 10.1 Introduction 437 | 10.2 Structure Definitions 437 | 10.2.1 Self-Referential Structures 438 | 10.2.2 Defining Variables of Structure Types 439 | 10.2.3 Structure Tag Names 439 | 10.2.4 Operations That Can Be Performed on Structures 439 | 10.3 Initializing Structures 440 | 10.4 Accessing Structure Members with . and -> 440 | 10.5 Using Structures with Functions 442 | 10.6 typedef 443 | 10.7 Example: High-Performance Card Shuffling and Dealing Simulation 443 | 10.8 Unions 446 | 10.8.1 Union Declarations 446 | 10.8.2 Operations That Can Be Performed on Unions 447 | 10.8.3 Initializing Unions in Declarations 447 | 10.8.4 Demonstrating Unions 447 | 10.9 Bitwise Operators 448 | 10.9.1 Displaying an Unsigned Integer in Bits 449 | 10.9.2 Making Function displayBits More Generic and Portable 451 | 10.9.3 Using the Bitwise AND, Inclusive OR, Exclusive OR and Complement Operators 452 | 10.9.4 Using the Bitwise Left- and Right-Shift Operators 455 | 10.9.5 Bitwise Assignment Operators 456 | 10.10 Bit Fields 457 | 10.10.1 Defining Bit Fields 457 | 10.10.2 Using Bit Fields to Represent a Card's Face, Suit and Color 458 | 10.10.3 Unnamed Bit Fields 460 | 10.11 Enumeration Constants 460 | 10.12 Anonymous Structures and Unions 462 | 10.13 Secure C Programming 462 |
|-----------------------------------------------------------------------|-----------------------|--------------------------------|----------------------------------------|--------------------------------------------------|--------------------------------|-----------------------------------------------------------|----------------------------------|----------------------------------------------------|------------------------------------------|------------------|--------------------------------------------------------------------------|-----------------|-------------------------------|-------------------------------------------------------|------------------------------------------------|---------------------------------|----------------------------|---------------------------------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------|-----------------------------------------|----------------------|---------------------------------|-------------------------------------------------------------------------|--------------------------------|---------------------------------|-------------------------------------------|--------------------------------|

|                                 |                       |                |
|---------------------------------|-----------------------|----------------|
| <b>11 C File Processing 473</b> | 11.1 Introduction 474 | 11.2 Files and |
|---------------------------------|-----------------------|----------------|

Streams 474 11.3 Creating a Sequential-Access File 475

11.3.1 Pointer to a FILE 477 11.3.2 Using fopen to Open the File 477

11.3.3 Using feof to Check for the End-of-File Indicator 477

## 14 Contents

11.3.4 Using fprintf to Write to the File 478 11.3.5 Using fclose to Close the File 478 11.3.6 File Open Modes 479

11.4 Reading Data from a Sequential-Access File 481 11.4.1 Resetting the File Position Pointer 482 11.4.2 Credit Inquiry Program 482

11.5 Random-Access Files 486 11.6 Creating a Random-Access File 486

11.7 Writing Data Randomly to a Random-Access File 488

11.7.1 Positioning the File Position Pointer with fseek 490 11.7.2 Error Checking 491 11.8 Reading Data from a Random-Access File 491 11.9 Case Study: Transaction-Processing Program 493 11.10 Secure C Programming 498

## 12 C Data Structures 509

12.1 Introduction 510 12.2 Self-Referential Structures 511 12.3 Dynamic Memory Allocation 511 12.4 Linked Lists 512

12.4.1 Function insert 518 12.4.2 Function delete 519 12.4.3 Function printList 521

12.5 Stacks 521 12.5.1 Function push 525 12.5.2 Function pop 526 12.5.3 Applications of Stacks 526

12.6 Queues 527 12.6.1 Function enqueue 531 12.6.2 Function dequeue 532

12.7 Trees 533 12.7.1 Function insertNode 536 12.7.2 Traversals: Functions inOrder, preOrder and postOrder 537 12.7.3 Duplicate Elimination 538 12.7.4 Binary Tree Search 538 12.7.5 Other Binary Tree Operations 538 12.8 Secure C Programming 538

## 13 C Preprocessor 550

13.1 Introduction 551 13.2 #include Preprocessor Directive 551 13.3 #define Preprocessor Directive: Symbolic Constants 552 13.4 #define Preprocessor Directive: Macros 553 13.4.1 Macro with One Argument 553  
[Contents 15](#)

13.4.2 Macro with Two Arguments 554 13.4.3 Macro Continuation Character 554 13.4.4 #undef Preprocessor Directive 554 13.4.5 Standard Library Functions and Macros 554 13.4.6 Do Not Place Expressions with Side Effects in Macros 555

13.5 Conditional Compilation 555 13.5.1 #if...#endif Preprocessor Directive 555 13.5.2 Commenting Out Blocks of Code with #if...#endif 555 13.5.3 Conditionally Compiling Debugging Code 556

13.6 #error and #pragma Preprocessor Directives 556 13.7 # and ## Operators 556 13.8 Line Numbers 557 13.9 Predefined Symbolic Constants 557 13.10 Assertions 558 13.11 Secure C Programming 558

## 14 Other C Topics 563

14.1 Introduction 564 14.2 Redirecting

I/O 564

|                                                                |     |                                                              |     |
|----------------------------------------------------------------|-----|--------------------------------------------------------------|-----|
| 14.2.1 Redirecting Input with <                                | 564 | 14.2.2 Redirecting Input with                                | 565 |
| 14.2.3 Redirecting Output                                      | 565 |                                                              |     |
| 14.3 Variable-Length Argument Lists                            | 565 | 14.4 Using Command-Line Arguments                            | 567 |
| 14.5 Compiling Multiple-Source-File Programs                   | 569 |                                                              |     |
| 14.5.1 extern Declarations for Global Variables in Other Files | 569 | 14.5.2 Function Prototypes                                   | 569 |
| 14.5.3 Restricting Scope with static                           | 570 | 14.5.4 Makefiles                                             | 570 |
| 14.6 Program Termination with exit and atexit                  | 570 | 14.7 Suffixes for Integer and Floating-Point Literals        | 572 |
| 14.8 Signal Handling                                           | 572 | 14.9 Dynamic Memory Allocation: Functions calloc and realloc | 575 |
| 14.10 Unconditional Branching with goto                        | 575 |                                                              |     |

## 15 C++ as a Better C; Introducing Object Technology 581

|                   |     |          |     |                                            |     |
|-------------------|-----|----------|-----|--------------------------------------------|-----|
| 15.1 Introduction | 582 | 15.2 C++ | 582 | 15.3 A Simple Program: Adding Two Integers | 583 |
|-------------------|-----|----------|-----|--------------------------------------------|-----|

|                                |     |                          |     |                      |     |                              |     |
|--------------------------------|-----|--------------------------|-----|----------------------|-----|------------------------------|-----|
| 15.3.1 Addition Program in C++ | 583 | 15.3.2 <iostream> Header | 584 | 15.3.3 main Function | 584 | 15.3.4 Variable Declarations | 584 |
| 16 Contents                    |     |                          |     |                      |     |                              |     |

|                                                                 |     |                                     |     |                        |     |                                    |     |                                              |     |                              |     |
|-----------------------------------------------------------------|-----|-------------------------------------|-----|------------------------|-----|------------------------------------|-----|----------------------------------------------|-----|------------------------------|-----|
| 15.3.5 Standard Output Stream and Standard Input Stream Objects | 584 | 15.3.6 std::endl Stream Manipulator | 585 | 15.3.7 std:: Explained | 585 | 15.3.8 Concatenated Stream Outputs | 585 | 15.3.9 return Statement Not Required in main | 585 | 15.3.10 Operator Overloading | 585 |
| 15.4 C++ Standard Library                                       | 586 | 15.5 Header Files                   | 586 | 15.6 Inline Functions  | 588 | 15.7 C++ Keywords                  | 590 | 15.8 References and Reference Parameters     | 591 |                              |     |

|                             |     |                                                    |     |                                                |     |                                              |     |                                                    |     |
|-----------------------------|-----|----------------------------------------------------|-----|------------------------------------------------|-----|----------------------------------------------|-----|----------------------------------------------------|-----|
| 15.8.1 Reference Parameters | 591 | 15.8.2 Passing Arguments by Value and by Reference | 592 | 15.8.3 References as Aliases within a Function | 594 | 15.8.4 Returning a Reference from a Function | 595 | 15.8.5 Error Messages for Uninitialized References | 596 |
|-----------------------------|-----|----------------------------------------------------|-----|------------------------------------------------|-----|----------------------------------------------|-----|----------------------------------------------------|-----|

|                            |     |                         |     |                                       |     |                            |     |                          |     |
|----------------------------|-----|-------------------------|-----|---------------------------------------|-----|----------------------------|-----|--------------------------|-----|
| 15.9 Empty Parameter Lists | 596 | 15.10 Default Arguments | 596 | 15.11 Unary Scope Resolution Operator | 598 | 15.12 Function Overloading | 599 | 15.13 Function Templates | 602 |
|----------------------------|-----|-------------------------|-----|---------------------------------------|-----|----------------------------|-----|--------------------------|-----|

|                                      |     |                                   |     |                                                     |     |                                          |     |                                                    |     |                                             |     |                                       |     |                                                                  |     |                                                               |     |                                     |     |                                                              |     |               |     |
|--------------------------------------|-----|-----------------------------------|-----|-----------------------------------------------------|-----|------------------------------------------|-----|----------------------------------------------------|-----|---------------------------------------------|-----|---------------------------------------|-----|------------------------------------------------------------------|-----|---------------------------------------------------------------|-----|-------------------------------------|-----|--------------------------------------------------------------|-----|---------------|-----|
| 15.13.1 Defining a Function Template | 602 | 15.13.2 Using a Function Template | 603 | 15.14 Introduction to Object Technology and the UML | 605 | 15.14.1 Basic Object Technology Concepts | 605 | 15.14.2 Classes, Data Members and Member Functions | 606 | 15.14.3 Object-Oriented Analysis and Design | 607 | 15.14.4 The Unified Modeling Language | 608 | 15.15 Introduction to C++ Standard Library Class Template vector | 608 | 15.15.1 Problems Associated with C-Style Pointer-Based Arrays | 608 | 15.15.2 Using Class Template vector | 609 | 15.15.3 Exception Handling: Processing an Out-of-Range Index | 613 | 15.16 Wrap-Up | 615 |
|--------------------------------------|-----|-----------------------------------|-----|-----------------------------------------------------|-----|------------------------------------------|-----|----------------------------------------------------|-----|---------------------------------------------|-----|---------------------------------------|-----|------------------------------------------------------------------|-----|---------------------------------------------------------------|-----|-------------------------------------|-----|--------------------------------------------------------------|-----|---------------|-----|

## 16 Introduction to Classes, Objects and Strings 621

|                   |     |                                              |     |                                                  |     |                                                                  |     |      |  |
|-------------------|-----|----------------------------------------------|-----|--------------------------------------------------|-----|------------------------------------------------------------------|-----|------|--|
| 16.1 Introduction | 622 | 16.2 Defining a Class with a Member Function | 622 | 16.3 Defining a Member Function with a Parameter | 625 | 16.4 Data Members, set Member Functions and get Member Functions | 629 | 16.5 |  |
|-------------------|-----|----------------------------------------------|-----|--------------------------------------------------|-----|------------------------------------------------------------------|-----|------|--|

## 17 Classes: A Deeper Look; Throwing Exceptions 659

17.1 Introduction 660 17.2 Time Class Case Study 661 17.3 Class Scope and Accessing Class Members 667 17.4 Access Functions and Utility Functions 668 17.5 Time Class Case Study: Constructors with Default Arguments 669 17.6 Destructors 675 17.7 When Constructors and Destructors Are Called 675 17.8 Time Class Case Study: A Subtle Trap—Returning a Reference or a Pointer to a private Data Member 679 17.9 Default Memberwise Assignment 682 17.10 `const` Objects and `const` Member Functions 684 17.11 Composition: Objects as Members of Classes 686 17.12 `friend` Functions and `friend` Classes 692 17.13 Using the `this` Pointer 694 17.14 `static` Class Members 700 17.15 Wrap-Up 705

## 18 Operator Overloading; Class `string` 715

18.1 Introduction 716 18.2 Using the Overloaded Operators of Standard Library Class `string` 717 18.3 Fundamentals of Operator Overloading 720 18.4 Overloading Binary Operators 721 18.5 Overloading the Binary Stream Insertion and Stream Extraction Operators 722 18.6 Overloading Unary Operators 726 18.7 Overloading the Unary Prefix and Postfix `++` and `--` Operators 727 18.8 Case Study: A `Date` Class 728 18.9 Dynamic Memory Management 733 18.10 Case Study: `Array` Class 735 18.10.1 Using the `Array` Class 736 18.10.2 `Array` Class Definition 740 18.11 Operators as Member vs. Non-Member Functions 748 18.12 Converting Between Types 748 18.13 `explicit` Constructors and Conversion Operators 750 18.14 Overloading the Function Call Operator `()` 752 18.15 Wrap-Up 753

## 19 Object-Oriented Programming: Inheritance 764

19.1 Introduction 765 19.2 Base Classes and Derived Classes 765 19.3 Relationship between Base and Derived Classes 768 19.3.1 Creating and Using a `CommissionEmployee` Class 768 19.3.2 Creating a `BasePlusCommissionEmployee` Class Without Using Inheritance 773

### 18 [Contents](#)

19.3.3 Creating a `CommissionEmployee`—`BasePlusCommissionEmployee` Inheritance Hierarchy 779 19.3.4 `CommissionEmployee`—`BasePlusCommissionEmployee` Inheritance Hierarchy Using `protected` Data 783 19.3.5 `CommissionEmployee`—`BasePlusCommissionEmployee` Inheritance Hierarchy Using `private` Data 786 19.4 Constructors and Destructors in Derived Classes 791 19.5 `public`, `protected` and `private` Inheritance 793 19.6 Software Engineering with Inheritance 794 19.7 Wrap-Up 794

## 20 Object-Oriented Programming: Polymorphism

|            |                                                                                      |     |                                                                                                                                                                              |     |                                                                      |     |
|------------|--------------------------------------------------------------------------------------|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|----------------------------------------------------------------------|-----|
| <b>799</b> | 20.1 Introduction                                                                    | 800 | 20.2 Introduction to Polymorphism: Polymorphic Video Game                                                                                                                    | 801 | 20.3 Relationships Among Objects in an Inheritance Hierarchy         | 801 |
|            | 20.3.1 Invoking Base-Class Functions from Derived-Class Objects                      | 802 | 20.3.2 Aiming Derived-Class Pointers at Base-Class Objects                                                                                                                   | 805 | 20.3.3 Derived-Class Member-Function Calls via Base-Class Pointers   | 806 |
|            | 20.3.4 Virtual Functions and Virtual Destructors                                     | 808 | 20.4 Type Fields and <code>switch</code> Statements                                                                                                                          | 815 | 20.5 Abstract Classes and Pure virtual Functions                     | 815 |
|            | 20.6 Case Study: Payroll System Using Polymorphism                                   | 817 | 20.6.1 Creating Abstract Base Class <code>Employee</code>                                                                                                                    | 818 | 20.6.2 Creating Concrete Derived Class <code>SalariedEmployee</code> | 822 |
|            | 20.6.3 Creating Concrete Derived Class <code>CommissionEmployee</code>               | 824 | 20.6.4 Creating Indirect Concrete Derived Class <code>BasePlusCommissionEmployee</code>                                                                                      | 826 | 20.6.5 Demonstrating Polymorphic Processing                          | 828 |
|            | 20.7 (Optional) Polymorphism, Virtual Functions and Dynamic Binding “Under the Hood” | 832 | 20.8 Case Study: Payroll System Using Polymorphism and Runtime Type Information with Downcasting, <code>dynamic_cast</code> , <code>typeid</code> and <code>type_info</code> | 835 | 20.9 Wrap-Up                                                         | 839 |

## 21 Stream Input/Output: A Deeper Look 844

|      |                                                                                           |     |                                                                                                                |     |                                                                                                              |     |
|------|-------------------------------------------------------------------------------------------|-----|----------------------------------------------------------------------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------------|-----|
| 21.1 | Introduction                                                                              | 845 | 21.2 Streams                                                                                                   | 846 | 21.2.1 Classic Streams vs. Standard Streams                                                                  | 846 |
|      | 21.2.2 <code>iostream</code> Library Headers                                              | 847 | 21.2.3 Stream Input/Output Classes and Objects                                                                 | 847 | 21.3 Stream Output                                                                                           | 849 |
|      | 21.3.1 Output of <code>char *</code> Variables                                            | 850 | 21.3.2 Character Output Using Member Function <code>put</code>                                                 | 850 | 21.4 Stream Input                                                                                            | 851 |
|      | 21.4.1 <code>get</code> and <code>getline</code> Member Functions                         | 851 | 21.4.2 <code>istream</code> Member Functions <code>peek</code> , <code>putback</code> and <code>ignore</code>  | 854 | 21.4.3 Type-Safe I/O                                                                                         | 854 |
|      | 21.5 Unformatted I/O Using <code>read</code> , <code>write</code> and <code>gcount</code> | 854 | 21.6 Introduction to Stream Manipulators                                                                       | 855 | 21.6.1 Integral Stream Base: <code>dec</code> , <code>oct</code> , <code>hex</code> and <code>setbase</code> | 856 |
|      | 21.6.2 Floating-Point Precision ( <code>precision</code> , <code>setprecision</code> )    | 856 | 21.6.3 Field Width ( <code>width</code> , <code>setw</code> )                                                  | 858 | 21.6.4 User-Defined Output Stream Manipulators                                                               | 859 |
|      | 21.7 Stream Format States and Stream Manipulators                                         | 860 | 21.7.1 Trailing Zeros and Decimal Points ( <code>showpoint</code> )                                            | 861 | 21.7.2 Justification ( <code>left</code> , <code>right</code> and <code>internal</code> )                    | 862 |
|      | 21.7.3 Padding ( <code>fill</code> , <code>setfill</code> )                               | 864 | 21.7.4 Integral Stream Base ( <code>dec</code> , <code>oct</code> , <code>hex</code> , <code>showbase</code> ) | 865 | 21.7.5 Floating-Point Numbers; Scientific and Fixed Notation                                                 | 866 |
|      | ( <code>scientific</code> , <code>fixed</code> )                                          | 866 | 21.7.6 Uppercase/Lowercase Control ( <code>uppercase</code> )                                                  | 867 | 21.7.7 Specifying Boolean Format ( <code>boolalpha</code> )                                                  | 867 |
|      | 21.7.8 Setting and Resetting the Format State via Member Function <code>flags</code>      | 868 | 21.8 Stream Error States                                                                                       | 869 | 21.9 Tying an Output Stream to an Input Stream                                                               | 872 |
|      | 21.10 Wrap-Up                                                                             | 872 |                                                                                                                |     |                                                                                                              |     |

[Contents 19](#)

## **22 Exception Handling: A Deeper Look 881** 22.1

Introduction 882 22.2 Example: Handling an Attempt to Divide by Zero 882  
22.3 Rethrowing an Exception 888 22.4 Stack Unwinding 889 22.5 When to  
Use Exception Handling 891 22.6 Constructors, Destructors and Exception  
Handling 892 22.7 Exceptions and Inheritance 893 22.8 Processing new  
Failures 893 22.9 Class `unique_ptr` and Dynamic Memory Allocation 896 22.10  
Standard Library Exception Hierarchy 899 22.11 Wrap-Up 900

## **23 Introduction to Custom Templates 906** 23.1

Introduction 907 23.2 Class Templates 907 23.3 Function Template to  
Manipulate a Class-Template Specialization Object 912 23.4 Nontype  
Parameters 914 23.5 Default Arguments for Template Type Parameters 914  
23.6 Overloading Function Templates 915 23.7 Wrap-Up 915  
20 [Contents](#)

## **A C and C++ Operator Precedence Charts 918 B**

## **ASCII Character Set 922**

## **C Number Systems 923** C.1 Introduction 924 C.2 Abbreviating

Binary Numbers as Octal and Hexadecimal Numbers 927 C.3 Converting  
Octal and Hexadecimal Numbers to Binary Numbers 928 C.4 Converting  
from Binary, Octal or Hexadecimal to Decimal 928 C.5 Converting from  
Decimal to Binary, Octal or Hexadecimal 929 C.6 Negative Binary Numbers:  
Two's Complement Notation 931

## **D Sorting: A Deeper Look 936** D.1 Introduction 937 D.2 Big

O Notation 937 D.3 Selection Sort 938 D.4 Insertion Sort 942 D.5 Merge  
Sort 945

## **E Multithreading and Other C11 and C99 Topics**

**956** E.1 Introduction 957 E.2 New C99 Headers 958 E.3 Designated  
Initializers and Compound Literals 959 E.4 Type `bool` 961 E.5 Implicit `int` in  
Function Declarations 963 E.6 Complex Numbers 964 E.7 Additions to the  
Preprocessor 965 E.8 Other C99 Features 966

E.8.1 Compiler Minimum Resource Limits 966 E.8.2 The `restrict`  
Keyword 967 E.8.3 Reliable Integer Division 967 E.8.4 Flexible Array  
Members 967 E.8.5 Relaxed Constraints on Aggregate Initialization  
968 E.8.6 Type Generic Math 968 E.8.7 Inline Functions 968 E.8.8  
Return Without Expression 969 E.8.9 `__func__` Predefined Identifier 969  
E.8.10 `va_copy` Macro 969

E.9 New Features in the C11 Standard 969 E.9.1 New C11 Headers 970  
E.9.2 Multithreading Support 970 E.9.3 `quick_exit` function 978

E.9.4 Unicode<sup>®</sup> Support 978 E.9.5 `_Noreturn` Function Specifier 978 E.9.6  
Type-Generic Expressions 978 E.9.7 Annex L: Analyzability and Undefined  
Behavior 979 E.9.8 Memory Alignment Control 979 E.9.9 Static Assertions  
979 E.9.10 Floating-Point Types 980 E.10 Web Resources 980

## Appendices on the Web 983 Index 984

Appendices F, G and H are PDF documents posted online at the book's  
Companion Website (located at [www.pearsonglobaleditions.com/deitel](http://www.pearsonglobaleditions.com/deitel)).

### F Using the Visual Studio Debugger

### G Using the GNU gdb Debugger

### H Using the Xcode Debugger

*This page intentionally left blank*

Welcome to the C programming language and to *C How to Program, Eighth Edition*! This book presents leading-edge computing technologies for college students, instructors and software-development professionals.

At the heart of the book is the Deitel signature “live-code approach”—we present concepts in the context of complete working programs, rather than in code snippets. Each code example is followed by one or more sample executions. Read the online Before You Begin section at

[http://www.deitel.com/books/cht8/cht8\\_BYB.pdf](http://www.deitel.com/books/cht8/cht8_BYB.pdf)

to learn how to set up your computer to run the hundreds of code examples.  
All the source code is available at

[www.pearsonglobaleditions.com/deitel](http://www.pearsonglobaleditions.com/deitel)

Use the source code we provide to run every program as you study it. We believe that this book and its support materials will give you an informative, challenging and entertaining introduction to C. As you read the book, if you have questions, send an e-mail to [deitel@deitel.com](mailto:deitel@deitel.com)—we'll respond promptly. For book updates, visit [www.deitel.com/books/cht8/](http://www.deitel.com/books/cht8/), join our social media communities:

- Facebook<sup>®</sup>—<http://facebook.com/DeitelFan>
- Twitter<sup>®</sup>—[@deitel](https://twitter.com/deitel)
- LinkedIn<sup>®</sup>—<http://linkedin.com/company/deitel-&-associates> •
- YouTube<sup>™</sup>—<http://youtube.com/DeitelTV>

- Google+™ —<http://google.com/+DeitelFan>

and register for the *Deitel® Buzz Online* e-mail newsletter at:

<http://www.deitel.com/newsletter/subscribe.html>

## New and Updated Features

Here are some key features of *C How to Program, 8/e*:

- **Integrated More Capabilities of the C11 and C99 standards.** Support for the C11 and C99 standards varies by compiler. Microsoft Visual C++ supports a subset of the features that were added to C in C99 and C11—primarily the features that are also required by the C++ standard. We incorporated several widely supported C11 and C99 features into the book's early chapters, as appropriate for introduc

### 24 Preface

tory courses and for the compilers we used in this book. Appendix E, Multi threading and Other C11 and C99 Topics, presents more advanced features (such as multithreading for today's increasingly popular multi-core architectures) and various other features that are not widely supported by today's C compilers.

- **All Code Tested on Linux, Windows and OS X.** We retested all the example and exercise code using GNU gcc on Linux, Visual C++ on Windows (in Visual Studio 2013 Community Edition) and LLVM in Xcode on OS X.
- **Updated Chapter 1.** The new Chapter 1 engages students with updated intriguing facts and figures to get them excited about studying computers and computer programming. The chapter includes current technology trends and hardware discussions, the data hierarchy, social networking and a table of business and technology publications and websites that will help you stay up to date with the latest technology news and trends. We've included updated test-drives that show how to run a command-line C program on Linux, Microsoft Windows and OS X. We also updated the discussions of the Internet and web, and the introduction to object technology.
- **Updated Coverage of C++ and Object-Oriented Programming.** We updated Chapters 15–23 on object-oriented programming in C++ with material from our textbook *C++ How to Program, 9/e*, which is up-to-date with the C++11 standard.
- **Updated Code Style.** We removed the spacing inside parentheses and square brackets, and toned down our use of comments a bit. We also added parentheses to certain compound conditions for clarity.
- **Variable Declarations.** Because of improved compiler support, we were able to move variable declarations closer to where they're first used and define for-loop counter-control variables in each for's initialization section.
- **Summary Bullets.** We removed the end-of-chapter terminology lists and updated the detailed section-by-section, bullet-list summaries

with **bolded** key terms and, for most, page references to their defining occurrences.

- **Use of Standard Terminology.** To help students prepare to work in industry worldwide, we audited the book against the C standard and upgraded our terminology to use C standard terms in preference to general programming terms.
- **Online Debugger Appendices.** We've updated the online GNU `gdb` and Visual C++<sup>®</sup> debugging appendices, and added an Xcode<sup>®</sup> debugging appendix.
- **Additional Exercises.** We updated various exercises and added some new ones, including one for the Fisher-Yates unbiased shuffling algorithm in Chapter 10.

## Other Features

Other features of *C How to Program*, 8/e include:

- **Secure C Programming Sections.** Many of the C chapters end with a Secure C Programming Section. We've also posted a Secure C Programming Resource Center at [www.deitel.com/SecureC/](http://www.deitel.com/SecureC/). For more details, see the section "A Note About Secure C Programming" on the next page.

[A Note About Secure C Programming](#) 25

- **Focus on Performance Issues.** C (and C++) are favored by designers of performance intensive systems such as operating systems, real-time systems, embedded systems and communications systems, so we focus intensively on performance issues.
- **"Making a Difference" Contemporary Exercises.** We encourage you to use computers and the Internet to research and solve significant problems. These exercises are meant to increase awareness of important issues the world is facing. We hope you'll approach them with your own values, politics and beliefs.
- **Sorting: A Deeper Look.** Sorting places data in order, based on one or more sort keys. We begin our sorting presentation in Chapter 6 with a simple algorithm— in Appendix D, we present a deeper look. We consider several algorithms and compare them with regard to their memory consumption and processor demands. For this purpose, we present a friendly introduction to Big O notation, which indicates how hard an algorithm may have to work to solve a problem. Through examples and exercises, Appendix D discusses the selection sort, insertion sort, recursive merge sort, recursive selection sort, bucket sort and recursive Quicksort. Sorting is an intriguing problem because different sorting techniques achieve the same final result but they can vary hugely in their consumption of memory, CPU time and other system resources.
- **Titled Programming Exercises.** Most of the programming exercises are titled to help instructors conveniently choose assignments appropriate for their students.

- **Order of Evaluation.** We caution the reader about subtle order of evaluation issues.
- **C++-Style // Comments.** We use the newer, more concise C++-style // comments in preference to C's older style /\*...\*/ comments.

## A Note About Secure C Programming

Throughout this book, we focus on C programming *fundamentals*. When we write each *How to Program* book, we search the corresponding language's standards document for the features that we feel novices need to learn in a first programming course, and features that professional programmers need to know to begin working in that language. We also cover computer-science and software-engineering fundamentals for novices—our core audience.

*Industrial-strength* coding techniques in any programming language are beyond the scope of an introductory textbook. For that reason, our Secure C Programming sections present some key issues and techniques, and provide links and references so you can continue learning.

Experience has shown that it's difficult to build industrial-strength systems that stand up to attacks from viruses, worms, etc. Today, via the Internet, such attacks can be instantaneous and global in scope. Software vulnerabilities often come from simple programming issues. Building security into software from the start of the development cycle can greatly reduce costs and vulnerabilities.

The CERT<sup>®</sup> Coordination Center ([www.cert.org](http://www.cert.org)) was created to analyze and respond promptly to attacks. CERT—the Computer Emergency Response Team—publishes and promotes secure coding standards to help C programmers and others implement industrial-strength systems that avoid the programming practices that leave systems vulnerable to attacks. The CERT standards evolve as new security issues arise.

26 Preface

We've upgraded our code (as appropriate for an introductory book) to conform to various CERT recommendations. If you'll be building C systems in industry, consider reading *The CERT C Secure Coding Standard, 2/e* (Robert Seacord, Addison-Wesley Professional, 2014) and *Secure Coding in C and C++, 2/e* (Robert Seacord, Addison-Wesley Professional, 2013). The CERT guidelines are available free online at

<https://www.securecoding.cert.org/confluence/display/seccode/CERT+C+Coding+Standard>

Mr. Seacord, a technical reviewer for the C portion of the last edition of this book, provided specific recommendations on each of our Secure C Programming sections. Mr. Seacord is the Secure Coding Manager at CERT at Carnegie Mellon University's Software Engineering Institute (SEI) and an adjunct professor in the Carnegie Mellon University School of Computer Science.

The Secure C Programming sections at the ends of Chapters 2–13 discuss many important topics, including:

- testing for arithmetic overflows
- using unsigned integer types
- the more secure functions in the C standard's Annex K

- the importance of checking the status information returned by standard-library functions
- range checking
- secure random-number generation
- array bounds checking
- preventing buffer overflows
- input validation
- avoiding undefined behaviors
- choosing functions that return status information vs. using similar functions that do not
- ensuring that pointers are always NULL or contain valid addresses
- using C functions vs. using preprocessor macros, and more.

## Web-Based Materials

- preventing buffer

The book's open access Companion Website

(<http://www.pearsonglobaleditions.com/deitel>) contains source code for all the code examples and the following appendices in PDF format:

- Appendix F, Using the Visual Studio Debugger

- Appendix G, Using the GNU gdb Debugger
- Appendix H, Using the Xcode Debugger

## Dependency Charts

Figures 1 and 2 on the next two pages show the dependencies among the chapters to help instructors plan their syllabi. *C How to Program, 8/e* is appropriate for CS1 and many CS2 courses, and for intermediate-level C and C++ programming courses. The C++ part of the book assumes that you've studied C Chapters 1–10.

## Teaching Approach

*C How to Program, 8/e*, contains a rich collection of examples. We focus on good software engineering, program clarity, preventing common errors, program portability and performance issues.

### Teaching Approach 27

#### Introduction

1 Introduction to Computers, the Internet and the Web

#### Intro to Programming 2

Intro to C Programming

#### Control Statements and Functions

3 Structured Program Development in C

4 C Program Control

5 C Functions

## C Chapter Dependency Chart

[Note: Arrows pointing into a chapter indicate that chapter's dependencies.]

### Streams and Files 9 C

Formatted Input/Output

11 C File Processing

## Arrays, Pointers and Strings

6 C Arrays

7 C Pointers

8 C Characters and Strings

## Aggregate Types

10 C Structures, Unions, Bit  
Manipulation and Enumerations

## Data Structures

Structures

## Other Topics, Multithreading and the C11 Standard

5.14–5.16 Recursion 12 C Data

13 C Preprocessor 14 Other C

dependency chart.

Topics **Fig. 1** | C chapter

E Multithreading and Other C11  
and C99 Topics  
D Sorting: A Deeper Look

**Syntax Shading.** For readability, we syntax shade the code, similar to the way most IDEs and code editors syntax color code. Our syntax-shading conventions are:

comments appear like this in gray

**keywords appear like this in dark blue**

**constants and literal values appear like this in light blue** all other code

appears in black

## C++ Chapter Dependency Chart

17 Classes: A Deeper Look;  
Throwing Exceptions

18 Operator Overloading;  
Class string

### Object-Based Programming

15 C++ as a Better C;

20 OOP:

### Object-Oriented Programming

19 OOP: Inheritance

|              |              |             |
|--------------|--------------|-------------|
| Polymorphism | 22 Exception | 23 Intro to |
| 21 Stream    | Handling: A  | Custom      |
| Input/Output | Deeper Look  | Templates   |

**Fig. 2** | C++ chapter dependency chart.

**Code Highlighting.** We place gray rectangles around the key code in each program.

**Using Fonts for Emphasis.** We place the key terms and the index's page reference for each defining occurrence in **bold colored** text for easy reference. We emphasize C program text in the Lucida font (for example, `int x = 5;`).

**Objectives.** Each chapter begins with a list of objectives.

**Illustrations/Figures.** Abundant flowcharts, tables, line drawings, UML diagrams (in the C++ chapters), programs and program outputs are included.

**Programming Tips.** We include programming tips to help you focus on important aspects of program development. These tips and practices represent the best we've gleaned from a combined eight decades of programming and teaching experience.

### Good Programming Practices

*The Good Programming Practices call attention to techniques that will help you produce programs that are clearer, more understandable and more maintainable.*

### Common Programming Errors

*Pointing out these Common Programming Errors reduces the likelihood that you'll make them.*

### Error-Prevention Tips

*These tips contain suggestions for exposing and removing bugs from your programs and for avoiding bugs in the first place.*

Software Used in C How to Program, 8/e 29

### Performance Tips

*These tips highlight opportunities for making your programs run faster or minimizing the amount of memory that they occupy.*



## Portability Tips

The Portability Tips help you write code that will run on a variety of platforms.



## Software Engineering Observations

The Software Engineering Observations highlight architectural and design issues that affect the construction of software systems, especially large-scale systems.

**Summary Bullets.** We present a detailed section-by-section, bullet-list summary of each chapter with **bolded** key terms. For easy reference, most of the key terms are followed by the page number of their defining occurrences.

**Self-Review Exercises and Answers.** Extensive self-review exercises and answers are included for self-study.

**Exercises.** Each chapter concludes with a substantial set of exercises including:

- simple recall of important terminology and concepts

- identifying the errors in code samples
- writing individual program statements
- writing small portions of C functions (and C++ member functions and classes)
- writing complete programs
- implementing major projects

**Index.** We've included an extensive index, which is especially helpful when you use the book as a reference. Defining occurrences of key terms are highlighted in the index with a **bold colored** page number.

## Software Used in *C How to Program, 8/e*

We tested the programs in *C How to Program, 8/e* using the following free compilers:

- GNU C and C++ (<http://gcc.gnu.org/install/binaries.html>), which are already installed on most Linux systems and can be installed on OS X and Windows systems.
- Microsoft's Visual C++ in Visual Studio 2013 Community edition, which you can download from <http://go.microsoft.com/?linkid=9863608>
- LLVM in Apple's Xcode IDE, which OS X users can download from the Mac App Store.

For other free C and C++ compilers, visit:

<http://www.thefreecountry.com/compilers/cpp.shtml>

<http://www.compilers.net/Dir/Compilers/CCpp.htm>

<http://www.freebyte.com/programming/cpp/#cppcompilers>

[http://en.wikipedia.org/wiki/List\\_of\\_compilers#C.2B.2B\\_compilers](http://en.wikipedia.org/wiki/List_of_compilers#C.2B.2B_compilers)

## 30 Preface

## Instructor Resources

The following supplements are available to *qualified instructors only* through Pearson Education's password-protected Instructor Resource Center

- **PowerPoint® slides** containing all the code and figures in the text, plus bulleted items that summarize key points.
- **Test Item File** of multiple-choice questions (approximately two per top-level book section)
- **Solutions Manual** with solutions to *most* (but not all) of the end-of-chapter exercises. Please check the Instructor Resource Center to determine which exercises have solutions.

**Please do not write to us requesting access to the Instructor Resource Center. Access is restricted to college instructors teaching from the book. Instructors may obtain access only through their Pearson representatives.** If you're not a registered faculty member, contact your Pearson representative.

**Solutions are *not* provided for “project” exercises.** Check out our Programming Projects Resource Center for lots of additional exercise and project possibilities (<http://www.deitel.com/ProgrammingProjects/>).

## Acknowledgments

We'd like to thank Abbey Deitel and Barbara Deitel for long hours devoted to this project. Abbey co-authored Chapter 1. We're fortunate to have worked with the dedicated team of publishing professionals at Pearson. We appreciate the guidance, savvy and energy of Tracy Johnson, Executive Editor, Computer Science. Kelsey Loanes and Bob Engelhardt did a marvelous job managing the review and production processes, respectively.

### C How to Program, 8/e Reviewers

We wish to acknowledge the efforts of our reviewers. Under tight deadlines, they scrutinized the text and the programs and provided countless suggestions for improving the presentation: Dr. Brandon Invergo (GNU/European Bioinformatics Institute), Danny Kalev (A Certified System Analyst, C Expert and Former Member of the C++ Standards Committee), Jim Hogg (Program Manager, C/C++ Compiler Team, Microsoft Corporation), José Antonio González Seco (Parliament of Andalusia), Sebnem Onsay (Special Instructor, Oakland University School of Engineering and Computer Science), Alan Bunning (Purdue University), Paul Clingan (Ohio State University), Michael Geiger (University of Massachusetts, Lowell), Jeonghwa Lee (Shippensburg University), Susan Mengel (Texas Tech University), Judith O'Rourke (SUNY at Albany) and Chen-Chi Shin (Radford University).

### Other Recent Editions Reviewers

William Albrecht (University of South Florida), Ian Barland (Radford University), Ed James Beckham (Altera), John Benito (Blue Pilot Consulting, Inc. and Convener of ISO WG14—the Working Group responsible for the C Programming Language Standard), Dr. John F. Doyle (Indiana University Southeast), Alireza Fazelpour (Palm Beach Community College), Mahesh Hariharan (Microsoft), Hemanth H.M. (Software Engineer at SonicWALL), Kevin Mark Jones (Hewlett Packard), Lawrence Jones (UGS Corp.), Don

[A Special Thank You to Brandon Invergo and Jim Hogg 31](#)

Kostuch (Independent Consultant), Vytautas Leonavicius (Microsoft), Xiaolong Li (Indiana State University), William Mike Miller (Edison Design Group, Inc.), Tom Rethard (The University of Texas at Arlington), Robert Seacord (Secure Coding Manager at SEI/ CERT, author of *The CERT C Secure Coding Standard* and technical expert for the international standardization working group for the programming language C), José Antonio González Seco (Parliament of Andalusia), Benjamin Seyfarth (University of Southern Mississippi), Gary Sibbitts (St. Louis Community College at Meramec), William Smith (Tulsa Community College) and Douglas Walls (Senior Staff Engineer, C compiler, Sun Microsystems—now part of Oracle).

## A Special Thank You to Brandon Invergo and Jim Hogg

We were privileged to have Brandon Invergo (GNU/European Bioinformatics Institute) and Jim Hogg (Program Manager, C/C++ Compiler Team, Microsoft Corporation) do full book reviews. They scrutinized the C portion of the book, providing numerous insights and constructive comments. The largest part of our audience uses either the GNU gcc compiler or Microsoft's Visual C++ compiler (which also compiles C). Brandon and Jim helped us ensure that our content was accurate for the GNU and Microsoft compilers, respectively. Their comments conveyed a love of software engineering, computer science and education that we share.

Well, there you have it! C is a powerful programming language that will help you write high-performance, portable programs quickly and effectively. It scales nicely into the realm of enterprise systems development to help organizations build their business-critical and mission-critical information systems. As you read the book, we would sincerely appreciate your comments, criticisms, corrections and suggestions for improving the text. Please address all correspondence—including questions—to:

[deitel@deitel.com](mailto:deitel@deitel.com)

We'll respond promptly, and post corrections and

clarifications on: [www.deitel.com/books/cht8/](http://www.deitel.com/books/cht8/)

We hope you enjoy working with *C How to Program, Eighth Edition* as much as we enjoyed writing it!

*Paul Deitel*

*Harvey Deitel*

## About the Authors

**Paul Deitel**, CEO and Chief Technical Officer of Deitel & Associates, Inc., is a graduate of MIT, where he studied Information Technology. Through Deitel & Associates, Inc., he has delivered hundreds of programming courses to industry clients, including Cisco, IBM, Siemens, Sun Microsystems, Dell, Lucent Technologies, Fidelity, NASA at the Kennedy Space Center, the National Severe Storm Laboratory, White Sands Missile Range, Hospital Sisters Health System, Rogue Wave Software, Boeing, SunGard Higher

Educa tion, Stratus, Cambridge Technology Partners, One Wave, Hyperion Software, Adra Sys tems, Entergy, CableData Systems, Nortel Networks, Puma, iRobot, Invensys and many

## 32 Preface

more. He and his co-author, Dr. Harvey M. Deitel, are the world's best-selling program ming-language textbook/professional book/video authors.

**Dr. Harvey M. Deitel**, Chairman and Chief Strategy Officer of Deitel & Associates, Inc., has 54 years of experience in the computer field. Dr. Deitel earned B.S. and M.S. degrees in electrical engineering from MIT and a Ph.D. in mathematics from Boston Uni versity (all with a focus on computing). He has extensive college teaching experience, including earning tenure and serving as the Chairman of the Computer Science Department at Boston College before founding Deitel & Associates in 1991 with his son, Paul Deitel. The Deitels' publications have earned international recognition, with translations published in Chinese, Korean, Japanese, German, Russian, Spanish, French, Polish, Italian, Portu guese, Greek, Urdu and Turkish. Dr. Deitel has delivered hundreds of programming courses to academic institutions, major corporations, government organizations and the military.

## About Deitel & Associates, Inc.

Deitel & Associates, Inc., founded by Paul Deitel and Harvey Deitel, is an internationally recognized authoring and corporate training organization, specializing in computer pro gramming languages, object technology, mobile app development and Internet and web software technology. The company's training clients include many of the world's largest companies, government agencies, branches of the military, and academic institutions. The company offers instructor-led training courses delivered at client sites worldwide on major programming languages and platforms, including C, C++, Java™, Android app develop ment, Swift™ and iOS® app development, Visual C#®, Visual Basic®, Visual C++®, Py thon®, object technology, Internet and web programming and a growing list of additional programming and software development courses.

Through its 40-year publishing partnership with Pearson/Prentice Hall, Deitel & Asso ciates, Inc., publishes leading-edge programming textbooks and professional books in print and popular e-book formats, and *LiveLessons* video courses (available on Safari Books Online and other video platforms). Deitel & Associates, Inc. and the authors can be reached at:

[deitel@deitel.com](mailto:deitel@deitel.com)

To learn more about Deitel's *Dive-Into*® *Series* Corporate Training curriculum deliv ered to groups of software engineers at client sites worldwide, visit:

<http://www.deitel.com/training>

To request a proposal for on-site, instructor-led training at your organization, e-mail [deitel@deitel.com](mailto:deitel@deitel.com).

Individuals wishing to purchase Deitel books and *LiveLessons* video

training can do so through [www.deitel.com](http://www.deitel.com). Bulk orders by corporations, the government, the military and academic institutions should be placed directly with Pearson. For more information, visit

<http://www.informit.com/store/sales.aspx>

Pearson would like to thank and acknowledge Arup Bhattacharjee, RCC Institute of Information Technology, Soumen Mukherjee, RCC Institute of Information Technology, and Saru Dhir, Amity University, for reviewing the Global Edition.

# Introduction to Computers, the Internet and the Web

## Objectives

In this chapter, you'll learn:

- Basic computer concepts.
  - The different types of programming languages.
  - The history of the C programming language.
  - The purpose of the C Standard Library.
  - The basics of object technology.
  - A typical C program development environment.
- To test-drive a C application in Windows, Linux and Mac OS X.
- Some basics of the Internet and the World Wide Web.

## 34 Chapter 1 Introduction to Computers, the Internet and the Web

### 1.1 Introduction

### 1.2 Hardware and Software

#### 1.2.1 Moore's Law

#### 1.2.2 Computer Organization

### 1.3 Data Hierarchy

### 1.4 Machine Languages, Assembly Languages and High-Level Languages

### 1.5 The C

### Programming Language

### 1.6 C

|                                                                  |  |
|------------------------------------------------------------------|--|
| Standard Library                                                 |  |
| 1.7 C++ and Other C-Based Languages                              |  |
| Technology                                                       |  |
| 1.8.1 The Automobile as an Object                                |  |
| 1.8.2 Methods and Classes                                        |  |
| 1.8.3 Instantiation                                              |  |
| 1.8.4 Reuse                                                      |  |
| 1.8.5 Messages and Method Calls                                  |  |
| 1.8.6 Attributes and Instance Variables                          |  |
| 1.8.7 Encapsulation and Information Hiding                       |  |
| 1.8.8 Inheritance                                                |  |
| 1.9 Typical C Program-Development Environment                    |  |
| 1.9.1 Phase 1: Creating a Program                                |  |
| 1.9.2 Phases 2 and 3: Preprocessing and Compiling a C Program    |  |
| 1.9.3 Phase 4: Linking                                           |  |
| 1.9.4 Phase 5: Loading                                           |  |
| 1.9.5 Phase 6: Execution                                         |  |
| 1.9.6 Problems That May Occur at Execution Time                  |  |
| 1.9.7 Standard Input, Standard Output and Standard Error Streams |  |
| 1.10 Test-Driving a C Application in Windows, Linux and Mac OS X |  |

|                                                                                 |  |
|---------------------------------------------------------------------------------|--|
| 1.10.1 Running a C Application from the Windows Command Prompt                  |  |
| 1.10.2 Running a C Application Using GNU C with Linux                           |  |
| 1.10.3 Running a C Application Using the Terminal on Mac OS X                   |  |
| 1.11 Operating Systems                                                          |  |
| 1.11.1 Windows—A Proprietary Operating System                                   |  |
| 1.11.2 Linux—An Open-Source Operating System                                    |  |
| 1.11.3 Apple's Mac OS X; Apple's iOS for iPhone®, iPad® and iPod Touch® Devices |  |
| 1.11.4 Google's Android                                                         |  |
| 1.12 The Internet and World Wide Web                                            |  |
| 1.12.1 The Internet: A Network of Networks                                      |  |
| 1.12.2 The World Wide Web: Making the Internet User-Friendly                    |  |
| 1.12.3 Web Services                                                             |  |
| 1.12.4 Ajax                                                                     |  |
| 1.12.5 The Internet of Things                                                   |  |
| 1.13 Some Key Software Terminology                                              |  |
| 1.14 Keeping Up-to-Date with Information Technologies                           |  |

Self-Review Exercises | Answers to Self-Review Exercises | Exercises | Making a Difference

## 1.1 Introduction

Welcome to C and C++! C is a concise yet powerful computer programming language that's appropriate for technically oriented people with little or no programming experience and for experienced programmers to use in building substantial software systems. *C How to Program, Eighth Edition*, is an effective learning tool for each of these audiences.

The core of the book emphasizes software engineering through the proven methodologies of *structured programming* in C and *object-oriented programming* in C++. The book presents hundreds of complete working programs and shows the outputs produced when those programs are run on a computer. We call this the “live-code approach.” All of these example programs may be downloaded from our website [www.deitel.com/books/cht8/](http://www.deitel.com/books/cht8/).

Most people are familiar with the exciting tasks that computers perform. Using this textbook, you'll learn how to command computers to perform those tasks. It's **software** (i.e., the instructions you write to command computers to perform **actions** and make **decisions**) that controls computers (often referred to as **hardware**).

## 1.2 Hardware and Software 35

## 1.2 Hardware and Software

Computers can perform calculations and make logical decisions phenomenally faster than human beings can. Many of today's personal computers can perform billions of calculations in one second—more than a human can perform in a lifetime. *Supercomputers* are already performing *thousands of trillions (quadrillions)* of instructions per second! China's National University of Defense Technology's Tianhe-2 supercomputer can perform over 33 quadrillion calculations per second (33.86 *petaflops*)!<sup>1</sup> To put that in perspective, *the Tianhe-2 supercomputer can perform in one second about 3 million calculations for every person on the planet!* And supercomputing “upper limits” are growing quickly.

Computers process data under the control of sequences of instructions called **computer programs**. These software programs guide the computer through ordered actions specified by people called computer **programmers**.

A computer consists of various devices referred to as hardware (e.g., the keyboard, screen, mouse, hard disks, memory, DVD drives and processing units). Computing costs are dropping dramatically, owing to rapid developments in hardware and software technologies. Computers that might have filled large rooms and cost millions of dollars decades ago are now inscribed on silicon chips smaller than a fingernail, costing perhaps a few dollars each. Ironically, silicon is one of the most abundant materials on Earth—it's an ingredient in common sand. Silicon-chip technology has made computing so economical that computers have become a commodity.

### 1.2.1 Moore's Law

Every year, you probably expect to pay at least a little more for most products and services. The opposite has been the case in the computer and communications fields, especially with regard to the hardware supporting these technologies. For many decades, hardware costs have fallen rapidly.

Every year or two, the capacities of computers have approximately *doubled* inexpensively. This remarkable trend often is called **Moore's Law**, named for the person who identified it in the 1960s, Gordon Moore, co-founder of Intel—the leading manufacturer of the processors in today's computers and embedded systems. Moore's Law and *related* observations apply especially to the amount of memory that computers have for programs, the amount of secondary storage (such as disk storage) they have to hold programs and data over longer periods of time, and their processor speeds—the speeds at which they *execute* their programs (i.e., do their work).

Similar growth has occurred in the communications field—costs have plummeted as enormous demand for communications *bandwidth* (i.e., information-carrying capacity) has attracted intense competition. We know of no other fields in which technology improves so quickly and costs fall so rapidly. Such phenomenal improvement is truly *fostering the Information Revolution*.

## 1.2.2 Computer Organization

Regardless of differences in *physical* appearance, computers can be envisioned as divided into various **logical units** or sections (Fig. 1.1).

**Input unit** This “receiving” section obtains information (data and computer programs) from **input devices** and places it at the disposal of the other units for processing. Most user input is entered into computers through keyboards, touch screens and mouse devices. Other forms of input include receiving voice commands, scanning images and barcodes, reading from secondary storage devices (such as hard drives, DVD drives, Blu-ray Disc™ drives and USB flash drives—also called “thumb drives” or “memory sticks”), receiving video from a webcam and having your computer receive information from the Internet (such as when you stream videos from YouTube® or download e-books from Amazon). Newer forms of input include position data from a GPS device, and motion and orientation information from an *accelerometer* (a device that responds to up/down, left/right and forward/backward acceleration) in a smart phone or game controller (such as Microsoft® Kinect® for Xbox®, Wii™ Remote and Sony® PlayStation® Move).

**Output unit** This “shipping” section takes information the computer has processed and places it on various **output devices** to make it available for use outside the computer. Most information that’s output from computers today is displayed on screens (including touch screens), printed on paper (“going green” discourages this), played as audio or video on PCs and media players (such as Apple’s iPods) and giant screens in sports stadiums, transmitted over the Internet or used to control other devices, such as robots and “intelligent” appliances. Information is also commonly output to secondary storage devices, such as hard drives, DVD drives and USB flash drives. Popular recent forms of output are smart phone and game controller vibration, and virtual reality devices like Oculus Rift.

**Memory unit** This rapid-access, relatively low-capacity “warehouse” section retains information that has been entered through the input unit, making it immediately available for processing when needed. The memory unit also retains processed information until it can be placed on output devices by the output unit. Information in the memory unit is *volatile*—it’s typically lost when the computer’s power is turned off. The memory unit is often called either **memory**, **primary memory** or **RAM** (Random Access Memory). Main memories on desktop and notebook computers contain as much as 128 GB of RAM, though 2 to 16 GB is most common. GB stands for gigabytes; a gigabyte is

approximately one billion bytes. A **byte** is eight bits. A bit is either a 0 or a 1.

### Arithmetic and logic unit (ALU)

This “manufacturing” section performs *calculations*, such as addition, subtraction, multiplication and division. It also contains the *decision* mechanisms that

allow the computer, for example, to compare two items from the memory unit to determine whether they’re equal. In today’s systems, the ALU is implemented as part of the next logical unit, the CPU.

**Fig. 1.1** | Logical units of a computer. (Part 1 of 2.)

## 1.3 Data Hierarchy 37

### Central processing unit (CPU)

### Secondary storage unit

This “administrative” section coordinates and supervises the operation of the other sections. The CPU tells the input unit when information should be read into the memory unit, tells the ALU when information from the memory unit should be used in calculations and tells the output unit when to send information from the memory unit to certain output devices.

Many of today’s computers have multiple CPUs and, hence, can perform many

operations simultaneously. A **multi-core processor** implements multiple processors on a single integrated-circuit chip—a *dual-core processor* has two CPUs and a *quad core processor* has four CPUs. Today’s desktop computers have processors that can execute billions of instructions per second.

This is the long-term, high-capacity “warehousing” section. Programs or data not actively being used by the other units normally are placed on secondary storage devices (e.g., your *hard drive*) until they’re again needed, possibly hours, days, months or even years later. Information on secondary storage devices is *persistent*—it’s preserved even when the computer’s power is turned off. Secondary storage information takes much longer to access than information in primary memory, but its cost per unit is much less. Examples of secondary storage devices include hard drives, DVD drives and USB flash drives, some of which can hold over 2 TB (TB stands for terabytes; a terabyte is approximately one trillion bytes). Typical hard drives on desktop and notebook computers hold up to 2 TB, and some desktop hard drives can hold up to 6 TB.

**Fig. 1.1** | Logical units of a computer. (Part 2 of 2.)

## 1.3 Data Hierarchy

Data items processed by computers form a **data hierarchy** that becomes larger and more complex in structure as we progress from the simplest data items (called “bits”) to richer ones, such as characters and fields. Figure 1.2 illustrates a portion of the data hierarchy.

### Bits

The smallest data item in a computer can assume the value 0 or the value 1. It’s called a **bit** (short for “binary digit”—a digit that can assume one of *two* values). Remarkably, the impressive functions performed by computers involve only the simplest manipulations of 0s and 1s—*examining a bit’s value*,

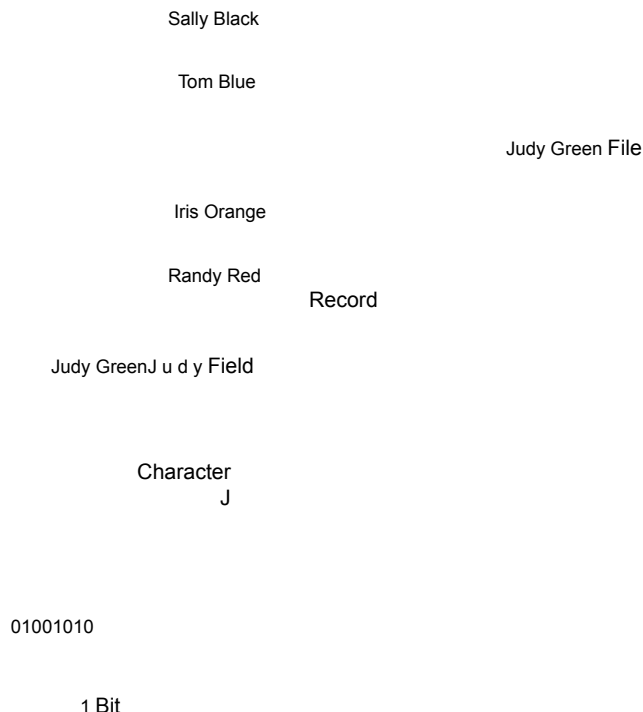
setting a bit's value and reversing a bit's value (from 1 to 0 or from 0 to 1).

## Characters

It's tedious for people to work with data in the low-level form of bits. Instead, they prefer to work with *decimal digits* (0–9), *letters* (A–Z and a–z), and *special symbols* (e.g., \$, @, %, &, \*, (, ), –, +, ", :, ? and /). Digits, letters and special symbols are known as **characters**. The computer's **character set** is the set of all the characters used to write programs and represent data items. Computers process only 1s and 0s, so a computer's character set represents every character as a pattern of 1s and 0s. C supports various character sets (including **Unicode**<sup>®</sup>) that are composed of characters containing one, two or four bytes (8, 16 or 32 bits). Unicode contains characters for many of the world's languages. See Appendix B for more information on

38 Chapter 1 Introduction to Computers, the Internet and the Web

the **ASCII (American Standard Code for Information Interchange)** character set—the popular subset of Unicode that represents uppercase and lowercase letters, digits and some common special characters.



**Fig. 1.2** | Data hierarchy.

## Fields

Just as characters are composed of bits, **fields** are composed of characters or bytes. A field is a group of characters or bytes that conveys meaning. For example, a field consisting of uppercase and lowercase letters can be used to represent a person's name, and a field consisting of decimal digits could represent a person's age.

## Records

Several related fields can be used to compose a **record**. In a payroll system, for example, the record for an employee might consist of the following fields (possible types for these fields are shown in parentheses):

- Employee identification number (a whole number)
- Name (a string of characters)
- Address (a string of characters)
- Hourly pay rate (a number with a decimal point)
- Year-to-date earnings (a number with a decimal point)
- Amount of taxes withheld (a number with a decimal point)

### 1.3 Data Hierarchy 39

Thus, a record is a group of related fields. In the preceding example, all the fields belong to the *same* employee. A company might have many employees and a payroll record for each.

## Files

A **file** is a group of related records. [Note: More generally, a file contains arbitrary data in arbitrary formats. In some operating systems, a file is viewed simply as a *sequence of bytes*— any organization of the bytes in a file, such as organizing the data into records, is a view created by the application programmer.] It's not unusual for an organization to have many files, some containing billions, or even trillions, of characters of information.

## Database

A **database** is a collection of data organized for easy access and manipulation. The most popular model is the *relational database*, in which data is stored in simple *tables*. A table includes *records* and *fields*. For example, a table of students might include first name, last name, major, year, student ID number and grade point average fields. The data for each student is a record, and the individual pieces of information in each record are the fields. You can *search*, *sort* and otherwise manipulate the data based on its relationship to multiple tables or databases. For example, a university might use data from the student database in combination with data from databases of courses, on-campus housing, meal plans, etc.

## Big Data

The amount of data being produced worldwide is enormous and growing quickly. According to IBM, approximately 2.5 quintillion bytes (2.5 *exabytes*) of data are created daily and 90% of the world's data was created in just the past two years.<sup>2</sup> According to an IDC study, the global data supply will reach 40 *zettabytes* (equal to 40 trillion gigabytes) annually by 2020.<sup>3</sup> Figure 1.3 shows some common byte measurements. **Big data** applications deal with massive amounts of data and this field is growing quickly, creating lots of opportunity for software developers. According to a study by Gartner Group, over 4 million IT jobs globally will support big data by 2015.<sup>4</sup>

1 kilobyte (KB) 1024 bytes  $10^3$  (1024 bytes exactly)  
 1 megabyte (MB) 1024 kilobytes  $10^6$  (1,000,000 bytes)  
 1 gigabyte (GB) 1024 megabytes  $10^9$  (1,000,000,000 bytes)  
 1 terabyte (TB) 1024 gigabytes  $10^{12}$  (1,000,000,000,000 bytes) 1 petabyte  
 (PB) 1024 terabytes  $10^{15}$  (1,000,000,000,000,000 bytes) 1 exabyte (EB) 1024  
 petabytes  $10^{18}$  (1,000,000,000,000,000,000 bytes) 1 zettabyte (ZB) 1024  
 exabytes  $10^{21}$  (1,000,000,000,000,000,000,000 bytes)

**Fig. 1.3** | Byte measurements.

2. [http://www.ibm.com/smarterplanet/us/en/business\\_analytics/article/it\\_business\\_intelligence.html](http://www.ibm.com/smarterplanet/us/en/business_analytics/article/it_business_intelligence.html).

3. <http://recode.net/2014/01/10/stuffed-why-data-storage-is-hot-again-really/>. 4.

<http://tech.fortune.cnn.com/2013/09/04/big-data-employment-boom/>.

40 Chapter 1 Introduction to Computers, the Internet and the Web

## 1.4 Machine Languages, Assembly Languages and High Level Languages

Programmers write instructions in various programming languages, some directly understandable by computers and others requiring intermediate *translation* steps. Hundreds of such languages are in use today. These may be divided into three general types: **1.** Machine languages

**2.** Assembly languages

**3.** High-level languages

### **Machine Languages**

Any computer can directly understand only its own **machine language**, defined by its hardware design. Machine languages generally consist of strings of numbers (ultimately reduced to 1s and 0s) that instruct computers to perform their most elementary operations one at a time. Machine languages are *machine dependent* (a particular machine language can be used on only one type of computer). Such languages are cumbersome for humans. For example, here's a section of an early machine-language payroll program that adds over time pay to base pay and stores the result in gross pay:

```
+1300042774
+1400593419
+1200274027
```

### **Assembly Languages and Assemblers**

Programming in machine language was simply too slow and tedious for most programmers. Instead of using the strings of numbers that computers could directly understand, programmers began using English-like abbreviations to represent elementary operations. These abbreviations formed the basis of **assembly languages**. *Translator programs* called **assemblers** were developed to convert early assembly-language programs to machine language at computer speeds. The following section of an assembly-language

payroll program also adds overtime pay to base pay and stores the result in gross pay:

```
load basepay
add overpay
store grosspay
```

Although such code is clearer to humans, it's incomprehensible to computers until translated to machine language.

### **High-Level Languages and Compilers**

With the advent of assembly languages, computer usage increased rapidly, but programmers still had to use numerous instructions to accomplish even the simplest tasks. To speed the programming process, **high-level languages** were developed in which single statements could be written to accomplish substantial tasks. Translator programs called **compilers** convert high-level language programs into machine language. High-level languages allow you to write instructions that look almost like everyday English and contain commonly used mathematical notations. A payroll program written in a high-level language might contain a *single* statement such as

```
grossPay = basePay + overTimePay
```

## 1.5 The C Programming Language 41

From the programmer's standpoint, high-level languages are preferable to machine and assembly languages. C is one of the most widely used high-level programming languages.

### **Interpreters**

Compiling a large high-level language program into machine language can take considerable computer time. *Interpreter* programs, developed to execute high-level language programs directly, avoid the delay of compilation, although they run slower than compiled programs.

## 1.5 The C Programming Language

C evolved from two previous languages, BCPL and B. BCPL was developed in 1967 by Martin Richards as a language for writing operating systems and compilers. Ken Thompson modeled many features in his B language after their counterparts in BCPL, and in 1970 he used B to create early versions of the UNIX operating system at Bell Laboratories.

The C language was evolved from B by Dennis Ritchie at Bell Laboratories and was originally implemented in 1972. C initially became widely known as the development language of the UNIX operating system. Many of today's leading operating systems are written in C and/or C++. C is mostly hardware independent—with careful design, it's possible to write C programs that are **portable** to most computers.

### **Built for Performance**

C is widely used to develop systems that demand performance, such as operating systems, embedded systems, real-time systems and communications systems (Figure 1.4).

Operating systems C's portability and performance make it desirable for implementing operating systems, such as Linux and portions of

Microsoft's Windows and Google's Android. Apple's OS X is built in Objective-C, which was derived from C. We discuss some key popular desktop/notebook operating systems and mobile operating systems in Section 1.11.

Embedded systems The vast majority of the microprocessors produced each year are embedded in devices other than general-purpose computers.

These **embedded systems** include navigation systems, smart home appliances, home security systems, smartphones, tablets, robots, intelligent traffic intersections and more. C is one of the most popular programming languages for developing embedded systems, which typically need to run as fast as possible and conserve memory. For example, a car's antilock brakes must respond immediately to slow or stop the car without skidding; game controllers used for video games should respond instantaneously to prevent any lag between the controller and the action in the game, and to ensure smooth animations.

**Fig. 1.4** | Some popular performance-oriented C applications. (Part 1 of 2.)  
42 Chapter 1 Introduction to Computers, the Internet and the Web

Real-time systems Real-time systems are often used for "mission-critical" applications that require nearly instantaneous and predictable response

times. Real-time systems need to work continuously—for example, an air-traffic-control system must constantly monitor the positions and velocities of the planes and report that information to air-traffic controllers without delay so that they can alert the planes to change course if there's a possibility of a collision.

Communications systems Communications systems need to route massive amounts of data to their destinations quickly to ensure that things such as audio and video are delivered smoothly and without delay.

**Fig. 1.4** | Some popular performance-oriented C applications. (Part 2 of 2.)

By the late 1970s, C had evolved into what's now referred to as "traditional C." The publication in 1978 of Kernighan and Ritchie's book, *The C Programming Language*, drew wide attention to the language. This became one of the most successful computer science books of all time.

### **Standardization**

The rapid expansion of C over various types of computers (sometimes called **hardware plat forms**) led to many variations that were similar but often incompatible. This was a serious problem for programmers who needed to develop code that would run on several platforms. It became clear that a standard version of C was needed. In 1983, the X3J11 technical committee was created under the American National Standards Committee on

Computers and Information Processing (X3) to “provide an unambiguous and machine-independent definition of the language.” In 1989, the standard was approved as ANSI X3.159-1989 in the United States through the **American National Standards Institute (ANSI)**, then worldwide through the **International Standards Organization (ISO)**. We call this simply Standard C. This standard was updated in 1999—its standards document is referred to as *INCITS/ISO/ IEC 9899-1999* and often referred to simply as C99. Copies may be ordered from the American National Standards Institute ([www.ansi.org](http://www.ansi.org)) at [webstore.ansi.org/ansidocstore](http://webstore.ansi.org/ansidocstore).

### **The C11 Standard**

We also discuss the latest C standard (referred to as C11), which was approved in 2011. C11 refines and expands C’s capabilities. We’ve integrated into the text and Appendix E (in easy-to-include-or-omit sections) many of the new features implemented in leading C compilers.

#### **Portability Tip 1.1**

*Because C is a hardware-independent, widely available language, applications written in C often can run with little or no modification on a wide range of computer systems.*

## **1.6 C Standard Library**

As you’ll learn in Chapter 5, C programs consist of pieces called **functions**. You can program all the functions that you need to form a C program, but most C programmers take

### **1.7 C++ and Other C-Based Languages 43**

advantage of the rich collection of existing functions called the **C Standard Library**. Thus, there are really two parts to learning how to program in C—learning the C language itself and learning how to use the functions in the C Standard Library. Throughout the book, we discuss many of these functions. P. J. Plauger’s book *The Standard C Library* is must reading for programmers who need a deep understanding of the library functions, how to implement them and how to use them to write portable code. We use and explain many C library functions throughout this text.

*C How to Program, 8/e* encourages a *building-block approach* to creating programs. Avoid “reinventing the wheel.” Instead, use existing pieces—this is called **software reuse**. When programming in C you’ll typically use the following building blocks:

- C Standard Library functions
- Functions you create yourself
- Functions other people (whom you trust) have created and made available to you

The advantage of creating your own functions is that you’ll know exactly how they work. You’ll be able to examine the C code. The disadvantage is the time-consuming effort that goes into designing, developing, debugging and performance-tuning new functions.

#### **Performance Tip 1.1**

*Using C Standard Library functions instead of writing your own versions can improve program performance, because these functions are carefully*

## Portability Tip 1.2

*Using C Standard Library functions instead of writing your own comparable versions can improve program portability, because these functions are used in virtually all Standard C implementations.*

## 1.7 C++ and Other C-Based Languages

C++ was developed by Bjarne Stroustrup at Bell Laboratories. It has its roots in C, providing a number of features that “spruce up” the C language. More important, it provides capabilities for **object-oriented programming**. **Objects** are essentially reusable software **components** that model items in the real world. Using a modular, object-oriented design-and-implementation approach can make software-development groups more productive. Chapters 15–23 present a condensed treatment of C++ selected from our book *C++ How to Program*. Figure 1.5 introduces several other popular C-based programming languages.

**Objective-C** Objective-C is an object-oriented language based on C. It was developed in the early 1980s and later acquired by NeXT, which in turn was acquired by Apple. It has become the key programming language for the OS X operating system and all iOS-powered devices (such as iPods, iPhones and iPads).

**Fig. 1.5 | Popular C-based programming languages. (Part 1 of 2.)**  
44 Chapter 1 Introduction to Computers, the Internet and the Web

**Java** Sun Microsystems in 1991 funded an internal corporate research project which resulted in the C++-based object-oriented programming language called Java. A key goal of Java is to enable the writing of programs that will run on a broad variety of computer systems and computer-controlled devices. This is sometimes called “write once, run anywhere.” Java is used to develop large-scale enterprise applications, to enhance the functionality of web servers (the computers that provide the content we see in our web browsers), to provide applications for consumer devices (smartphones, television set-top boxes and more) and for many other purposes. Java is also the language of Android app development.

**C#** Microsoft’s three primary object-oriented programming languages are Visual Basic (based on the original Basic), Visual C++ (based on C++) and Visual C# (based on C++ and Java, and developed for integrating the Internet and the web into computer applications). Non-Microsoft versions of C# are also available.

**PHP** PHP, an object-oriented, open-source scripting language supported by a community of users and developers, is used by millions of websites. PHP is platform independent—implementations exist for all major UNIX, Linux, Mac and Windows operating systems. PHP also supports many databases, including the popular open-source MySQL.

**Python** Python, another object-oriented scripting language, was released publicly in

1991. Developed by Guido van Rossum of the National Research Institute for Mathematics and Computer Science in Amsterdam (CWI), Python draws heavily from Modula-3—a systems programming language. Python is “extensible”—it can be extended through classes and programming interfaces.

JavaScript JavaScript is the most widely used scripting language. It’s primarily used to add dynamic behavior to web pages—for example, animations and improved interactivity with the user. It’s provided with all major web browsers.

Swift Swift, Apple’s new programming language for developing iOS and Mac apps, was announced at the Apple World Wide Developer Conference (WWDC) in June 2014. Although apps can still be developed and maintained with Objective-C, Swift is Apple’s app-development language of the future. It’s a modern language that eliminates some of the complexity of Objective-C, making it easier for beginners and those transitioning from other high-level languages such as Java, C#, C++ and C. Swift emphasizes performance and security, and has full access to the iOS and Mac programming capabilities.

**Fig. 1.5** | Popular C-based programming languages. (Part 2 of 2.)

## 1.8 Object Technology

This section is intended for readers who will be studying C++ in the later part of this book. Building software quickly, correctly and economically remains an elusive goal at a time when demands for new and more powerful software are soaring. *Objects*, or more precisely the *classes* objects come from, are essentially *reusable* software components. There are date objects, time objects, audio objects, video objects, automobile objects, people objects, etc. Almost any *noun* can be reasonably represented as a software object in terms of *attributes*

### 1.8 Object Technology 45

(e.g., name, color and size) and *behaviors* (e.g., calculating, moving and communicating). Software developers are discovering that using a modular, object-oriented design-and implementation approach can make software-development groups much more productive than was possible with earlier techniques—object-oriented programs are often easier to understand, correct and modify.

### 1.8.1 The Automobile as an Object

To help you understand objects and their contents, let’s begin with a simple analogy. Suppose you want to *drive a car and make it go faster by pressing its accelerator pedal*. What must happen before you can do this? Well, before you can drive a car, someone has to *design* it. A car typically begins as engineering drawings, similar to the *blueprints* that describe the design of a house. These drawings include the design for an accelerator pedal. The pedal *hides* from the driver the complex mechanisms that actually make the car go faster, just as the brake pedal “hides” the mechanisms that slow the car, and the steering wheel “hides” the mechanisms that turn the car. This enables people with little or no knowledge of how engines, braking and steering mechanisms work to drive a car easily.

Just as you cannot cook meals in the kitchen of a blueprint, you cannot drive a car’s engineering drawings. Before you can drive a car, it must be

*built* from the engineering drawings that describe it. A completed car has an *actual* accelerator pedal to make it go faster, but even that's not enough—the car won't accelerate on its own (hopefully!), so the driver must *press* the pedal to accelerate the car.

### 1.8.2 Methods and Classes

Let's use our car example to introduce some key object-oriented programming concepts. Performing a task in a program requires a **method**. The method houses the program state *ments* that actually perform its tasks. It hides these statements from its user, just as a car's accelerator pedal hides from the driver the mechanisms of making the car go faster. In object-oriented programming languages, we create a program unit called a **class** to house the set of methods that perform the class's tasks. For example, a class that represents a bank account might contain one method to *deposit* money to an account, another to *withdraw* money from an account and a third to *inquire* what the account's current balance is. A class is similar in concept to a car's engineering drawings, which house the design of an accelerator pedal, steering wheel, and so on.

### 1.8.3 Instantiation

Just as someone has to *build a car* from its engineering drawings before you can actually drive a car, you must *build an object* of a class before a program can perform the tasks that the class's methods define. The process of doing this is called *instantiation*. An object is then referred to as an **instance** of its class.

### 1.8.4 Reuse

Just as a car's engineering drawings can be *reused* many times to build many cars, you can *reuse* a class many times to build many objects. Reuse of existing classes when building new classes and programs saves time and effort. Reuse also helps you build more reliable and

46 [Chapter 1 Introduction to Computers, the Internet and the Web](#)

effective systems, because existing classes and components often have undergone extensive *testing*, *debugging* and *performance* tuning. Just as the notion of *interchangeable parts* was crucial to the Industrial Revolution, reusable classes are crucial to the software revolution that has been spurred by object technology.

#### Software Engineering Observation 1.1

*Use a building-block approach to creating your programs. Avoid reinventing the wheel— use existing high-quality pieces wherever possible. Such software reuse is a key benefit of object-oriented programming.*

### 1.8.5 Messages and Method Calls

When you drive a car, pressing its gas pedal sends a *message* to the car to perform a task— that is, to go faster. Similarly, you *send messages to an object*. Each message is implemented as a **method call** that tells a method



of the object to perform its task. For example, a program might call a particular bank-account object's *deposit* method to increase the account's balance.

### 1.8.6 Attributes and Instance Variables

A car, besides having capabilities to accomplish tasks, also has *attributes*, such as its color, its number of doors, the amount of gas in its tank, its current speed and its record of total miles driven (i.e., its odometer reading). Like its capabilities, the car's attributes are represented as part of its design in its engineering diagrams (which, for example, include an odometer and a fuel gauge). As you drive an actual car, these attributes are carried along with the car. Every car maintains its *own* attributes. For example, each car knows how much gas is in its own gas tank, but *not* how much is in the tanks of *other* cars.

An object, similarly, has attributes that it carries along as it's used in a program. These attributes are specified as part of the object's class. For example, a bank-account object has a *balance attribute* that represents the amount of money in the account. Each bank account object knows the balance in the account it represents, but *not* the balances of the *other* accounts in the bank. Attributes are specified by the class's **instance variables**.

### 1.8.7 Encapsulation and Information Hiding

Classes (and their objects) **encapsulate**, i.e., encase, their attributes and methods. A class's (and its objects) attributes and methods are intimately related. Objects may communicate with one another, but they're normally not allowed to know how other objects are implemented—implementation details are *hidden* within the objects themselves. This **information hiding**, as we'll see, is crucial to good software engineering.

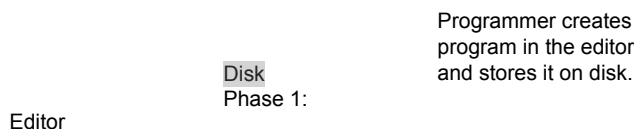
### 1.8.8 Inheritance

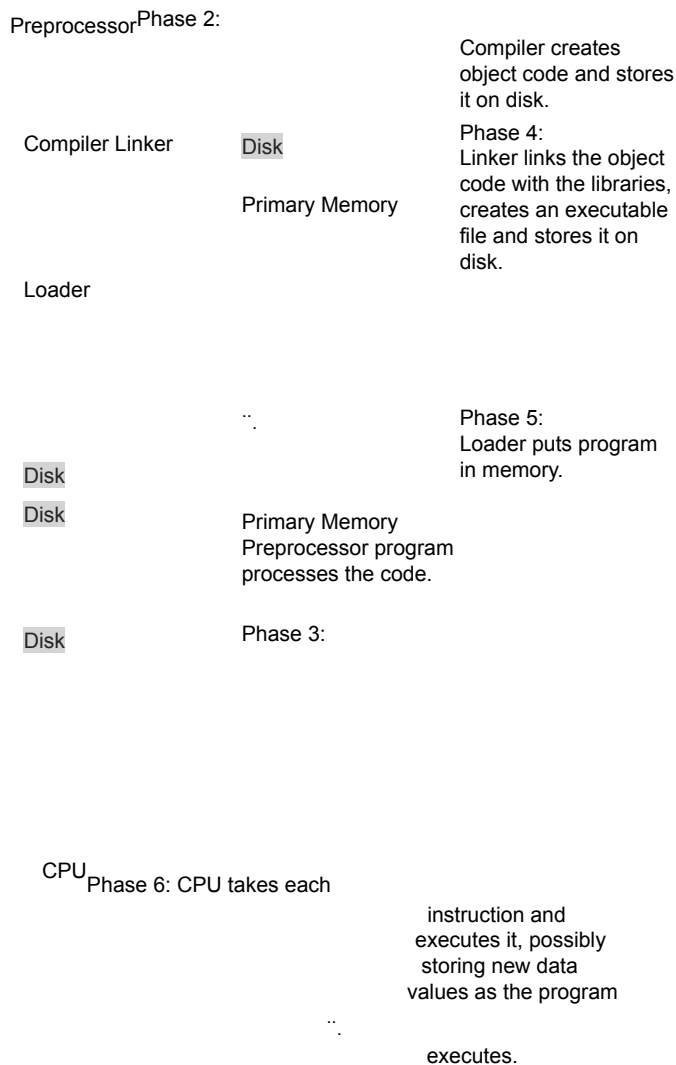
A new class of objects can be created conveniently by **inheritance**—the new class (called the **subclass**) starts with the characteristics of an existing class (called the **superclass**), possibly customizing them and adding unique characteristics of its own. In our car analogy, an object of class “convertible” certainly *is an* object of the more *general* class “automobile,” but more *specifically*, the roof can be raised or lowered.

## 1.9 Typical C Program-Development Environment 47

### 1.9 Typical C Program-Development Environment

C systems generally consist of several parts: a program-development environment, the language and the C Standard Library. The following discussion explains the typical C development environment shown in Fig. 1.6.





**Fig. 1.6 | Typical C development environment.**

48 Chapter 1 Introduction to Computers, the Internet and the Web

C programs typically go through six phases to be executed (Fig. 1.6). These are: **edit**, **preprocess**, **compile**, **link**, **load** and **execute**. Although *How to Program, 8/e*, is a generic C textbook (written independently of the details of any particular operating system), we concentrate in this section on a typical Linux-based C system. [Note: The programs in this book will run with little or no modification on most current C systems, including Microsoft Windows-based systems.] If you're not using a Linux system, refer to the documentation for your system or ask your instructor how to accomplish these tasks in your environment. Check out our C Resource Center at [www.deitel.com/C](http://www.deitel.com/C) to locate "getting started" tutorials for popular C compilers and development environments.

## 1.9.1 Phase 1: Creating a Program

Phase 1 consists of editing a file. This is accomplished with an **editor program**. Two editors widely used on Linux systems are `vi` and `emacs`. Software packages for the C/C++ integrated program development environments such as Eclipse and Microsoft Visual Studio have editors that are integrated into the programming environment. You type a C program with the editor, make corrections if necessary, then store the program on a secondary storage device such as a hard disk. C program filenames should end with the `.c` extension.

## 1.9.2 Phases 2 and 3: Preprocessing and Compiling a C

**Program** In Phase 2, you give the command to **compile** the program. The compiler translates the C program into machine-language code (also referred to as **object code**). In a C system, a **pre processor** program executes automatically before the compiler's translation phase begins. The **C preprocessor** obeys special commands called **preprocessor directives**, which indicate that certain manipulations are to be performed on the program before compilation. These manipulations usually consist of including other files in the file to be compiled and performing various text replacements. The most common preprocessor directives are discussed in the early chapters; a detailed discussion of preprocessor features appears in Chapter 13.

In Phase 3, the compiler translates the C program into machine-language code. A **syntax error** occurs when the compiler cannot recognize a statement because it violates the rules of the language. The compiler issues an error message to help you locate and fix the incorrect statement. The C Standard does not specify the wording for error messages issued by the compiler, so the error messages you see on your system may differ from those on other systems. Syntax errors are also called **compile errors**, or **compile-time errors**.

## 1.9.3 Phase 4: Linking

The next phase is called **linking**. C programs typically contain references to functions defined elsewhere, such as in the standard libraries or in the private libraries of groups of programmers working on a particular project. The object code produced by the C compiler typically contains “holes” due to these missing parts. A **linker** links the object code with the code for the missing functions to produce an **executable image** (with no missing pieces). On a typical Linux system, the command to compile and link a program is called **gcc** (the GNU C compiler). To compile and link a program named `welcome.c`, type

```
gcc welcome.c
```

at the Linux prompt and press the *Enter* key (or *Return* key). [Note: Linux commands are case sensitive; make sure that each `c` is lowercase and that the letters in the filename are in

## 1.10 Test-Driving a C Application in Windows, Linux and Mac OS X 49

the appropriate case.] If the program compiles and links correctly, a file called `a.out` (by default) is produced. This is the executable image of our `welcome.c`

program.

### 1.9.4 Phase 5: Loading

The next phase is called **loading**. Before a program can be executed, the program must first be placed in memory. This is done by the **loader**, which takes the executable image from disk and transfers it to memory. Additional components from shared libraries that support the program are also loaded.

### 1.9.5 Phase 6: Execution

Finally, the computer, under the control of its CPU, **executes** the program one instruction at a time. To load and execute the program on a Linux system, type `./a.out` at the Linux prompt and press *Enter*.

### 1.9.6 Problems That May Occur at Execution Time

Programs do not always work on the first try. Each of the preceding phases can fail because of various errors that we'll discuss. For example, an executing program might attempt to divide by zero (an illegal operation on computers just as in arithmetic). This would cause the computer to display an error message. You would then return to the edit phase, make the necessary corrections and proceed through the remaining phases again to determine that the corrections work properly.



#### Common Programming Error 1.1

*Errors such as division-by-zero occur as a program runs, so they are called runtime errors or execution-time errors. Divide-by-zero is generally a fatal error, i.e., one that causes the program to terminate immediately without successfully performing its job. Nonfatal errors allow programs to run to completion, often producing incorrect results.*

### 1.9.7 Standard Input, Standard Output and Standard Error

**Streams** Most C programs input and/or output data. Certain C functions take their input from `stdin` (the **standard input stream**), which is normally the keyboard, but `stdin` can be re directed to another stream. Data is often output to `stdout` (the **standard output stream**), which is normally the computer screen, but `stdout` can be redirected to another stream. When we say that a program prints a result, we normally mean that the result is displayed on a screen. Data may be output to devices such as disks and printers. There's also a **standard error stream** referred to as `stderr`. The `stderr` stream (normally connected to the screen) is used for displaying error messages. It's common to route regular output data, i.e., `stdout`, to a device other than the screen while keeping `stderr` assigned to the screen so that the user can be immediately informed of errors.

## 1.10 Test-Driving a C Application in Windows, Linux and Mac OS X

In this section, you'll run and interact with your first C application. You'll begin by running a guess-the-number game, which randomly picks a number from 1 to 1000 and

prompts you to guess it. If your guess is correct, the game ends. If your guess is not correct, the application indicates it's higher or lower than the correct number. There's no limit on the number of guesses you can make but you should be able to guess any of the numbers in this range correctly in 10 or fewer tries. There's some nice computer science behind this game—in Section 6.10, Searching Arrays, you'll explore the *binary search* technique.

For this test-drive only, we've modified this application from the exercise you'll be asked to create in Chapter 5. Normally this application *randomly* selects the correct answers. The modified application uses the same sequence of correct answers every time you execute the program (though the particular sequence may vary by compiler), so you can use the same guesses we use in this section and see the same results.

We'll demonstrate running a C application using the Windows Command Prompt, a *shell* on Linux and a Terminal window in Mac OS X. The application runs similarly on all three platforms. After you perform the test-drive for your platform, you can try the *randomized* version of the game, which we've provided with each test drive's version of the example in a subfolder named `randomized_version`.

Many development environments are available in which you can compile, build and run C applications, such as GNU C, Dev C++, Microsoft Visual C++, CodeLite, Net Beans, Eclipse, Xcode, etc. Consult your instructor for information on your specific development environment. Most C++ development environments can compile both C and C++ programs.

In the following steps, you'll run the application and enter various numbers to guess the correct number. The elements and functionality that you see in this application are typical of those you'll learn to program in this book. We use fonts to distinguish between features you see on the screen (e.g., the Command Prompt) and elements that are not directly related to the screen. We emphasize screen features like titles and menus (e.g., the File menu) in a semibold sans-serif Helvetica font, and to emphasize filenames, text displayed by an application and values you should enter into an application (e.g., `Guess Number Or 500`) we use a sans-serif Lucida font. As you've noticed, the **defining occurrence** of each key term is set in **bold blue** type.

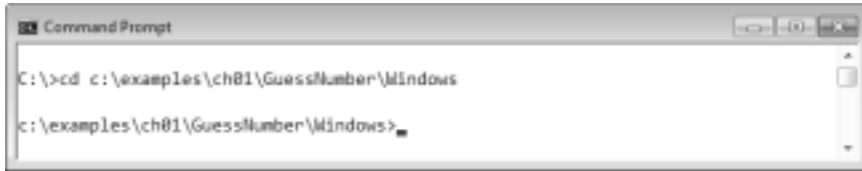
For the Windows version of the test drive in this section, we've modified the background color of the Command Prompt window to make the Command Prompt windows more readable. To modify the Command Prompt colors on your system, open a Command Prompt by selecting `Start > All Programs > Accessories > Command Prompt`, then right click the title bar and select Properties. In the "Command Prompt" Properties dialog box that appears, click the Colors tab, and select your preferred text and background colors.

### 1.10.1 Running a C Application from the Windows Command

**Prompt 1. Checking your setup.** It's important to read the Before You Begin section at [www.deitel.com/books/chtp8/](http://www.deitel.com/books/chtp8/) to make sure that you've copied the book's examples to your hard drive correctly.

**2. Locating the completed application.** Open a Command Prompt window. To change to the directory for the completed GuessNumber application, type `cd C:\examples\ch01\GuessNumber\Windows`, then press

*Enter* (Fig. 1.7). The command `cd` is used to change directories.  
1.10 Test-Driving a C Application in Windows, Linux and Mac OS X 51



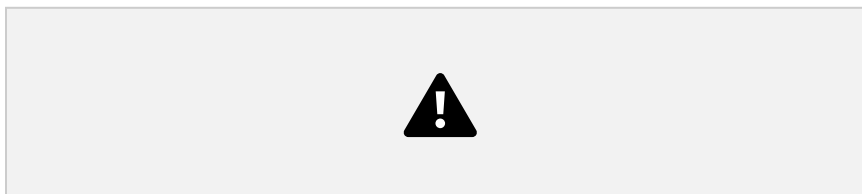
**Fig. 1.7 |** Opening a Command Prompt window and changing the directory.

3. **Running the *GuessNumber* application.** Now that you are in the directory that contains the *GuessNumber* application, type the command *GuessNumber* (Fig. 1.8) and press *Enter*. [Note: *GuessNumber.exe* is the actual name of the application; however, Windows assumes the *.exe* extension by default.]



**Fig. 1.8 |** Running the *GuessNumber* application.

4. **Entering your first guess.** The application displays "Please type your first guess.", then displays a question mark (?) as a prompt on the next line (Fig. 1.8). At the prompt, enter 500 (Fig. 1.9).



**Fig. 1.9 |** Entering your first guess.

5. **Entering another guess.** The application displays "Too high. Try again.", meaning that the value you entered is greater than the number the application chose as the correct guess. So, you should enter a lower number for your next guess. At the prompt, enter 250 (Fig. 1.10). The application again displays "Too high. Try again.", because the value you entered is still greater than the number that the application chose.



**Fig. 1.10** | Entering a second guess and receiving feedback.

- 6. Entering additional guesses.** Continue to play the game by entering values until you guess the correct number. The application will display "Excellent! You guessed the number!" (Fig. 1.11).



**Fig. 1.11** | Entering additional guesses and guessing the correct number.

- 7. Playing the game again or exiting the application.** After you guess correctly, the application asks if you'd like to play another game (Fig. 1.11). At the prompt, entering 1 causes the application to choose a new number and displays the message "Please type your first guess." followed by a question-mark prompt (Fig. 1.12), so you can make your first guess in the new game. Entering 2 ends the application and returns you to the application's directory at the Command Prompt (Fig. 1.13). Each time you execute this application from the beginning (i.e., *Step 3*), it will choose the same numbers for you to guess.

- 8. Close the Command Prompt window.**

1.10 Test-Driving a C Application in Windows, Linux and Mac OS X





Fig. 1.13 | Exiting the game.

### 1.10.2 Running a C Application Using GNU C with Linux

For the figures in this section, we use a bold font to point out the user input required by each step. In this test drive, we assume that you know how to copy the examples into your home directory. Please see your instructor if you have any questions regarding copying the files to your Linux system. Also, for the figures in this section, we use a bold font to point out the user input required by each step. The prompt in the shell on our system uses the tilde (~) character to represent the home directory, and each prompt ends with the dollar sign (\$) character. The prompt will vary among Linux systems.

1. **Checking your setup.** It's important to read the Before You Begin section at [www.deitel.com/books/cht8/](http://www.deitel.com/books/cht8/) to make sure that you've copied the book's examples to your hard drive correctly.
2. **Locating the completed application.** From a Linux shell, change to the completed GuessNumber application directory (Fig. 1.14) by typing

```
cd examples/ch01/GuessNumber/GNU
```

then pressing **Enter**. The command `cd` is used to change directories.

```
~$ cd examples/ch01/GuessNumber/GNU
~/examples/ch01/GuessNumber/GNU$
```

Fig. 1.14 | Changing to the GuessNumber application's directory.

54 Chapter 1 Introduction to Computers, the Internet and the Web

3. **Compiling the *GuessNumber* application.** To run an application on the GNU C++ compiler, you must first compile it by typing

```
gcc GuessNumber.c -o GuessNumber
```

as in Fig. 1.15. This command compiles the application. The `-o` option is followed by the name you'd like the executable file to have—`GuessNumber`.

```
~/examples/ch01/GuessNumber/GNU$ gcc -std=c11 GuessNumber.c -o GuessNumber
~/examples/ch01/GuessNumber/GNU$
```

Fig. 1.15 | Compiling the GuessNumber application using the `gcc` command.

4. **Running the *GuessNumber* application.** To run the executable file

GuessNumber, type `./GuessNumber` at the next prompt, then press *Enter* (Fig. 1.16).

```
~/examples/ch01/GuessNumber/GNU$ ./GuessNumber
```

```
I have a number between 1 and 1000.  
Can you guess my number?  
Please type your first guess.  
?
```

**Fig. 1.16** | Running the GuessNumber application.

**5. Entering your first guess.** The application displays "Please type your first guess.", then displays a question mark (?) as a prompt on the next line (Fig. 1.16). At the prompt, enter 500 (Fig. 1.17).

```
~/examples/ch01/GuessNumber/GNU$ ./GuessNumber
```

```
I have a number between 1 and 1000.  
Can you guess my number?  
Please type your first guess.  
? 500  
Too high. Try again.  
?
```

**Fig. 1.17** | Entering an initial guess.

**6. Entering another guess.** The application displays "Too high. Try again.", meaning that the value you entered is greater than the number the application chose as the correct guess (Fig. 1.17). At the next prompt, enter 250 (Fig. 1.18). This time the application displays "Too low. Try again.", because the value you entered is less than the correct guess.

## 1.10 Test-Driving a C Application in Windows, Linux and Mac OS X 55

```
~/examples/ch01/GuessNumber/GNU$ ./GuessNumber
```

```
I have a number between 1 and 1000.  
Can you guess my number?  
Please type your first guess.  
? 500  
Too high. Try again.  
? 250  
Too low. Try again.  
?
```

**Fig. 1.18** | Entering a second guess and receiving feedback.

**7. Entering additional guesses.** Continue to play the game (Fig. 1.19) by entering values until you guess the correct number. When you guess correctly, the application displays "Excellent! You guessed the number!"

```
Too low. Try again.  
? 375  
Too low. Try again.  
? 437  
Too high. Try again.  
? 406  
Too high. Try again.  
? 391  
Too high. Try again.  
? 383  
Too low. Try again.  
? 387  
Too high. Try again.  
? 385  
Too high. Try again.  
? 384
```

```
Excellent! You guessed the number!  
Would you like to play again?  
Please type ( 1=yes, 2=no )?
```

**Fig. 1.19** | Entering additional guesses and guessing the correct number.

**8. *Playing the game again or exiting the application.*** After you guess the correct number, the application asks if you'd like to play another game. At the prompt, entering 1 causes the application to choose a new number and displays the message "Please type your first guess." followed by a question-mark prompt (Fig. 1.20) so that you can make your first guess in the new game. Entering 2 ends the application and returns you to the application's directory in the shell (Fig. 1.21). Each time you execute this application from the beginning (i.e., *Step 4*), it will choose the same numbers for you to guess.

## 56 [Chapter 1 Introduction to Computers, the Internet and the Web](#)

```
Excellent! You guessed the number!  
Would you like to play again?  
Please type ( 1=yes, 2=no )? 1  
  
I have a number between 1 and 1000.  
Can you guess my number?  
Please type your first guess.  
?
```

**Fig. 1.20** | Playing the game again.

```
Excellent! You guessed the number!  
Would you like to play again?  
Please type ( 1=yes, 2=no )? 2  
  
~/examples/ch01/GuessNumber/GNU$
```

**Fig. 1.21** | Exiting the game.

### 1.10.3 Running a C Application Using the Terminal on Mac OS

**X** For the figures in this section, we use a bold font to point out the user input required by each step. You'll use Mac OS X's Terminal window to perform this test-drive. To open a Terminal window, click the Spotlight Search icon in the upper-right corner of your screen, then type `Terminal` to locate the Terminal application. Under Applications in the Spotlight Search results, select Terminal to open a Terminal window. The prompt in a Terminal window has the form `hostName:~ userFolder$` to represent your user directory. For the figures in this section, we removed the `hostName:` part and used the generic name `userFolder` to represent your user account's folder.

1. **Checking your setup.** It's important to read the Before You Begin section at [www.deitel.com/books/cht8/](http://www.deitel.com/books/cht8/) to make sure that you've copied the book's examples to your hard drive correctly. We assume that the examples are located in your user account's Documents/examples folder.
2. **Locating the completed application.** In the Terminal window, change to the completed GuessNumber application directory (Fig. 1.22) by typing

```
cd Documents/examples/ch01/GuessNumber/GNU
```

then pressing *Enter*. The command `cd` is used to change directories.

```
hostName:~ userFolder$ cd Documents/examples/ch01/GuessNumber/GNU
hostName:GNU$
```

**Fig. 1.22** | Changing to the GuessNumber application's directory.

3. **Compiling the *GuessNumber* application.** To run an application, you must first compile it by typing

```
clang GuessNumber.c -o GuessNumber
```

### 1.10 Test-Driving a C Application in Windows, Linux and Mac OS X 57

as in Fig. 1.23. This command compiles the application and produces an executable file called `GuessNumber`.

```
hostName:GNU~ userFolder$ clang GuessNumber.c -o GuessNumber
hostName:GNU~ userFolder$
```

**Fig. 1.23** | Compiling the GuessNumber application using the `gcc` command.

4. **Running the *GuessNumber* application.** To run the executable file `GuessNumber`, type `./GuessNumber` at the next prompt, then press *Enter* (Fig. 1.24).

```
hostName:GNU~ userFolder$ ./GuessNumber
```

```
I have a number between 1 and 1000.
```

```
Can you guess my number?
```

```
Please type your first guess.
```

```
?
```

**Fig. 1.24** | Running the GuessNumber application.

**5. Entering your first guess.** The application displays "Please type your first guess.", then displays a question mark (?) as a prompt on the next line (Fig. 1.24). At the prompt, enter 500 (Fig. 1.25).

```
hostName:GNU~ userFolder$ ./GuessNumber
```

```
I have a number between 1 and 1000.
```

```
Can you guess my number?
```

```
Please type your first guess.
```

```
? 500
```

```
Too low. Try again.
```

```
?
```

**Fig. 1.25** | Entering an initial guess.

**6. Entering another guess.** The application displays "Too low. Try again." (Fig. 1.25), meaning that the value you entered is less than the number the application chose as the correct guess. At the next prompt, enter 750 (Fig. 1.26). Again the application displays "Too low. Try again.", because the value you entered is less than the correct guess.

**7. Entering additional guesses.** Continue to play the game (Fig. 1.27) by entering values until you guess the correct number. When you guess correctly, the application displays "Excellent! You guessed the number!"

## 58 Chapter 1 Introduction to Computers, the Internet and the Web

```
hostName:GNU~ userFolder$ ./GuessNumber
```

```
I have a number between 1 and 1000.
```

```
Can you guess my number?
```

```
Please type your first guess.
```

```
? 500
```

```
Too low. Try again.
```

```
? 750
```

```
Too low. Try again.
```

```
?
```

**Fig. 1.26** | Entering a second guess and receiving feedback.

```
? 825
Too high. Try again.
? 788
Too low. Try again.
? 806
Too low. Try again.
? 815
Too high. Try again.
? 811
Too high. Try again.
? 808

Excellent! You guessed the number!
Would you like to play again?
Please type ( 1=yes, 2=no )?
```

**Fig. 1.27** | Entering additional guesses and guessing the correct number.

**8. *Playing the game again or exiting the application.*** After you guess the correct number, the application asks if you'd like to play another game. At the prompt, entering 1 causes the application to choose a new number and displays the message "Please type your first guess." followed by a question-mark prompt (Fig. 1.28) so you can make your first guess in the new game. Entering 2 ends the application and returns you to the application's folder in the Terminal window (Fig. 1.29). Each time you execute this application from the beginning (i.e., *Step 4*), it will choose the same numbers for you to guess.

```
Excellent! You guessed the number!
Would you like to play again?
Please type ( 1=yes, 2=no )? 1

I have a number between 1 and 1000.
Can you guess my number?
Please type your first guess.
?
```

**Fig. 1.28** | Playing the game again.

```
Excellent! You guessed the number!
Would you like to play again?
Please type ( 1=yes, 2=no )? 2

hostName:GNU~ userFolder$
```

**Fig. 1.29** | Exiting the game.

## 1.11 Operating Systems

**Operating systems** are software systems that make using computers more convenient for users, application developers and system administrators. They

provide services that allow each application to execute safely, efficiently and *concurrently* (i.e., in parallel) with other applications. The software that contains the core components of the operating system is the **kernel**. Popular desktop operating systems include Linux, Windows and Mac OS X. Popular mobile operating systems used in smartphones and tablets include Google's Android, Apple's iOS (for its iPhone, iPad and iPod Touch devices), Windows Phone and BlackBerry OS.

### 1.11.1 Windows—A Proprietary Operating System

In the mid-1980s, Microsoft developed the **Windows operating system**, consisting of a graphical user interface built on top of DOS—an enormously popular personal-computer operating system that users interacted with by typing commands. Windows borrowed many concepts (such as icons, menus and windows) popularized by early Apple Macintosh operating systems and originally developed by Xerox PARC. Windows 8.1 is Microsoft's latest operating system—its features include PC and tablet support, a tiles-based user interface, security enhancements, touch-screen and multi-touch support, and more. Windows is a *proprietary* operating system—it's controlled by Microsoft exclusively. It's by far the world's most widely used operating system.

### 1.11.2 Linux—An Open-Source Operating System

The **Linux** operating system—which is popular in servers, personal computers and embedded systems—is perhaps the greatest success of the *open-source* movement. The **open source software** development style departs from the *proprietary* development style (used, for example, with Microsoft's Windows and Apple's Mac OS X). With open-source development, individuals and companies—often worldwide—contribute their efforts in developing, maintaining and evolving software. Anyone can use and customize it for their own purposes, typically at no charge.

Some organizations in the open-source community are the *Eclipse Foundation* (the Eclipse Integrated Development Environment helps programmers conveniently develop software), the *Mozilla Foundation* (creators of the *Firefox web browser*), the *Apache Software Foundation* (creators of the *Apache web server* that delivers web pages over the Internet in response to web-browser requests) and *GitHub* and *SourceForge* (which provide the tools for managing open-source projects).

Rapid improvements to computing and communications, decreasing costs and open source software have made it easier and more economical to create software-based busi-

60 Chapter 1 Introduction to Computers, the Internet and the Web

nesses now than just a few decades ago. Facebook, which was launched from a college dorm room, was built with open-source software.<sup>5</sup>

A variety of issues—such as Microsoft's market power, the relatively small number of user-friendly Linux applications and the diversity of Linux distributions (Red Hat Linux, Ubuntu Linux and many others)—have prevented widespread Linux use on desktop computers. But Linux has become extremely popular on servers and in embedded systems, such as Google's Android-based smartphones.

### 1.11.3 Apple's Mac OS X; Apple's iOS for iPhone®, iPad® and iPod Touch® Devices

Apple, founded in 1976 by Steve Jobs and Steve Wozniak, quickly became a leader in personal computing. In 1979, Jobs and several Apple employees visited Xerox PARC (Palo Alto Research Center) to learn about Xerox's desktop computer that featured a graphical user interface (GUI). That GUI served as the inspiration for the Apple Macintosh, launched with much fanfare in a memorable Super Bowl ad in 1984.

The Objective-C programming language, created by Brad Cox and Tom Love at Stepstone in the early 1980s, added capabilities for object-oriented programming (OOP) to the C programming language. Steve Jobs left Apple in 1985 and founded NeXT Inc. In 1988, NeXT licensed Objective-C from StepStone and developed an Objective-C compiler and libraries which were used as the platform for the NeXTSTEP operating system's user interface and Interface Builder—used to construct graphical user interfaces.

Jobs returned to Apple in 1996 when Apple bought NeXT. Apple's Mac OS X operating system is a descendant of NeXTSTEP. Apple's proprietary operating system, **iOS**, is derived from Apple's Mac OS X and is used in the iPhone, iPad and iPod Touch devices.

### 1.11.4 Google's Android

**Android**—the fastest growing tablet and smartphone operating system—is based on the Linux kernel and Java as its primary programming language. One benefit of developing Android apps is the openness of the platform. The operating system is open source and free.

The Android operating system was developed by Android, Inc., which was acquired by Google in 2005. In 2007, the Open Handset Alliance™—which has 87 company members worldwide—was formed to develop, maintain and evolve Android, driving innovation in mobile technology and improving the user experience while reducing costs. As of April 2013, more than 1.5 million Android devices (smartphones, tablets, etc.) were being activated worldwide *daily*.<sup>6</sup> Android devices now include smartphones, tablets, e-readers, robots, jet engines, NASA satellites, game consoles, refrigerators, televisions, cameras, health-care devices, smartwatches, automobile in-vehicle infotainment systems (for controlling the radio, GPS, phone calls, thermostat, etc.) and more.<sup>7</sup> Android also executes on desktop and notebook computers.<sup>8</sup>

5. <https://code.facebook.com/projects/>.

6. <http://www.technobuffalo.com/2013/04/16/google-daily-android-activations-1-5-million/>.

7. <http://www.businessweek.com/articles/2013-05-29/behind-the-internet-of-things-is-android-and-its-everywhere>.

8. <http://www.android-x86.org>.

## 1.12 The Internet and World Wide Web

In the late 1960s, ARPA—the Advanced Research Projects Agency of the

United States Department of Defense—rolled out plans for networking the main computer systems of approximately a dozen ARPA-funded universities and research institutions. The computers were to be connected with communications lines operating at speeds on the order of 50,000 bits per second, a stunning rate at a time when most people (of the few who even had networking access) were connecting over telephone lines to computers at a rate of 110 bits per second. Academic research was about to take a giant leap forward. ARPA proceeded to implement what quickly became known as the ARPANET, the precursor to today's **Internet**. Today's fastest Internet speeds are on the order of billions of bits per second with trillion-bits-per-second speeds on the horizon!

Things worked out differently from the original plan. Although the ARPANET enabled researchers to network their computers, its main benefit proved to be the capability for quick and easy communication via what came to be known as electronic mail (e mail). This is true even on today's Internet, with e-mail, instant messaging, file transfer and social media such as Facebook and Twitter enabling billions of people worldwide to communicate quickly and easily.

The protocol (set of rules) for communicating over the ARPANET became known as the **Transmission Control Protocol (TCP)**. TCP ensured that messages, consisting of sequentially numbered pieces called *packets*, were properly routed from sender to receiver, arrived intact and were assembled in the correct order.

### 1.12.1 The Internet: A Network of Networks

In parallel with the early evolution of the Internet, organizations worldwide were implementing their own networks for both intraorganization (that is, within an organization) and interorganization (that is, between organizations) communication. A huge variety of networking hardware and software appeared. One challenge was to enable these different networks to communicate with each other. ARPA accomplished this by developing the Internet Protocol (IP), which created a true “network of networks,” the current architecture of the Internet. The combined set of protocols is now called **TCP/IP**.

Businesses rapidly realized that by using the Internet, they could improve their operations and offer new and better services to their clients. Companies started spending large amounts of money to develop and enhance their Internet presence. This generated fierce competition among communications carriers and hardware and software suppliers to meet the increased infrastructure demand. As a result, **bandwidth**—the information-carrying capacity of communications lines—on the Internet has increased tremendously, while hardware costs have plummeted.

### 1.12.2 The World Wide Web: Making the Internet User-Friendly

The **World Wide Web** (simply called “the web”) is a collection of hardware and software associated with the Internet that allows computer users to locate and view multimedia based documents (documents with various combinations of text, graphics, animations, audios and videos) on almost any subject. The introduction of the web was a relatively recent event. In 1989, Tim Berners-Lee of CERN (the European Organization for Nuclear Research) began to develop a technology for sharing information via

uments. Berners-Lee called his invention the **HyperText Markup Language (HTML)**. He also wrote communication protocols such as **HyperText Transfer Protocol (HTTP)** to form the backbone of his new hypertext information system, which he referred to as the World Wide Web.

In 1994, Berners-Lee founded an organization, called the **World Wide Web Consortium (W3C)**, (<http://www.w3.org>), devoted to developing web technologies. One of the W3C's primary goals is to make the web universally accessible to everyone regardless of disabilities, language or culture.

### 1.12.3 Web Services

**Web services** are software components stored on one computer that can be accessed by an app (or other software component) on another computer over the Internet. With web services, you can create *mashups*, which enable you to rapidly develop apps by combining complementary web services, often from multiple organizations and possibly other forms of information feeds. For example, 100 Destinations (<http://www.100destinations.co.uk>) combines the photos and tweets from Twitter with the mapping capabilities of Google Maps to allow you to explore countries around the world through the photos of others.

Programmableweb (<http://www.programmableweb.com/>) provides a directory of over 11,150 APIs and 7,300 mashups, plus how-to guides and sample code for creating your own mashups. Figure 1.30 lists some popular web services. According to Programmableweb, the three most widely used APIs for mashups are Google Maps, Twitter and YouTube.

| Web Service                                                    | Relative Popularity (approximate bar length) |
|----------------------------------------------------------------|----------------------------------------------|
| Google Maps Mapping services                                   | Very High                                    |
| Twitter Microblogging                                          | High                                         |
| YouTube Video search                                           | High                                         |
| Facebook Social networking                                     | Medium-High                                  |
| Instagram Photo sharing                                        | Medium                                       |
| Foursquare Mobile check-in                                     | Medium                                       |
| LinkedIn Social networking for business                        | Medium                                       |
| Groupon Social commerce                                        | Medium-Low                                   |
| Netflix Movie rentals                                          | Low-Medium                                   |
| eBay Internet auctions                                         | Low-Medium                                   |
| Wikipedia Collaborative encyclopedia                           | Low                                          |
| PayPal Payments                                                | Low                                          |
| Last.fm Internet radio                                         | Low                                          |
| Amazon eCommerce Shopping for books and lots of other products | Low                                          |
| Salesforce.com Customer Relationship Management (CRM)          | Low                                          |
| Skype                                                          | Low                                          |
| Internet telephony                                             | Low                                          |

**Fig. 1.30** | Some popular web services (<http://www.programmableweb.com/category/all/apis>). (Part 1 of 2.)

Microsoft Bing Search  
 Flickr Photo sharing  
 Zillow Real-estate pricing  
 Yahoo Search Search  
 WeatherBug Weather

**Fig. 1.30** | Some popular web services (<http://www.programmableweb.com/category/all/apis>). (Part 2 of 2.)

Figure 1.31 lists directories where you'll find information about many of the most popular web services. Figure 1.32 lists a few popular web mashups.

ProgrammableWeb [www.programmableweb.com](http://www.programmableweb.com)

Google Code API Directory [code.google.com/apis/gdata/docs/directory.html](http://code.google.com/apis/gdata/docs/directory.html) **Fig. 1.31**

| [Web services directories.](#)

<http://twikle.com/> Twikle uses Twitter web services to aggregate popular news stories being shared online.

<http://trendsmap.com/> TrendsMap uses Twitter and Google Maps. It allows you to track tweets by location and view them on a map in real time.

<http://www.coindesk.com/price/bitcoin-price-ticker-widget/> The Bitcoin Price Ticker Widget uses CoinDesk's APIs to display the real-time Bitcoin price, the day's high and low prices and a graph of the price fluctuations over the last sixty minutes.

<http://www.dutranslation.com/> The Double Translation mashup allows you to use Bing and Google translation services simultaneously to translate text to and from over 50 languages. You can then compare the results between the two.

<http://musicupdated.com/> Music Updated uses Last.fm and YouTube web services. Use it to track album releases, concert information and more for your favorite artists.

**Fig. 1.32** | A few popular web mashups.

**Ajax** technology helps Internet-based applications perform like desktop applications—a difficult task, given that such applications suffer transmission delays as data is shuttled back and forth between your computer and server computers on the Internet. Using Ajax, applications like Google Maps have achieved excellent performance and approach the look-and-feel of desktop applications.

### 1.12.5 The Internet of Things

The Internet is no longer just a network of computers—it's an **Internet of Things**. A *thing* is any object with an IP address and the ability to send data automatically over the Internet—e.g., a car with a transponder for paying tolls, a heart monitor implanted in a human, a smart meter that reports energy usage, mobile apps that can track your movement and location, and smart thermostats that adjust room temperatures based on weather forecasts and activity in the home.

## 1.13 Some Key Software Terminology

Figure 1.33 lists a number of buzzwords that you'll hear in the software development community.

|                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Agile software development                                                                                                                                                                                                                   | implemented faster and using fewer resources. Check out the Agile Alliance ( <a href="http://www.agilealliance.org">www.agilealliance.org</a> ) and the Agile Manifesto ( <a href="http://www.agilemanifesto.org">www.agilemanifesto.org</a> ). |
| <b>Agile software development</b> is a set of methodologies that try to get software                                                                                                                                                         |                                                                                                                                                                                                                                                 |
| Refactoring <b>Refactoring</b> involves reworking programs to make them clearer and easier to maintain while preserving their correctness and functionality. It's                                                                            |                                                                                                                                                                                                                                                 |
| widely employed with agile development methodologies. Many IDEs contain built-in <i>refactoring tools</i> to do major portions of the reworking automatically.                                                                               |                                                                                                                                                                                                                                                 |
| Design patterns <b>Design patterns</b> are proven architectures for constructing flexible and maintainable object-oriented software. The field of design patterns tries                                                                      |                                                                                                                                                                                                                                                 |
| to enumerate those recurring patterns, encouraging software designers to <i>reuse</i> them to develop better-quality software using less time, money and effort.                                                                             |                                                                                                                                                                                                                                                 |
| LAMP <b>LAMP</b> is an acronym for the open-source technologies that many developers use to build web applications inexpensively—it stands for                                                                                               |                                                                                                                                                                                                                                                 |
| <i>Linux</i> ,                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                 |
| <i>Apache</i> , <i>MySQL</i> and <i>PHP</i> (or <i>Perl</i> or <i>Python</i> —two other popular scripting languages). <i>MySQL</i> is an open-source database-management system.                                                             |                                                                                                                                                                                                                                                 |
| <i>PHP</i> is the most popular open-source server-side “scripting” language for developing web applications. <i>Apache</i> is the most popular web server software. The equivalent for Windows development is <i>WAMP</i> — <i>Windows</i> , |                                                                                                                                                                                                                                                 |
| <i>Apache</i> , <i>MySQL</i> and <i>PHP</i> .                                                                                                                                                                                                |                                                                                                                                                                                                                                                 |

**Fig. 1.33** | Software technologies. (Part 1 of 2.)

## 1.13 Some Key Software Terminology

65

### Software as a Service (SaaS)

### Platform as a Service (PaaS)

Software has generally been viewed as a product; most software still is offered this way. If you want to run an application, you

Cloud computing SaaS and PaaS are examples of **cloud computing**. You can use software and data stored in the “cloud”—i.e., accessed on remote computers (or servers) via the Internet and available on demand—rather than having it stored locally on your desktop, notebook computer or mobile device. This allows you to increase or decrease computing resources to meet your needs at any given time, which is more cost effective than purchasing hardware to provide enough storage and processing power to meet occasional peak demands. Cloud computing also saves money by shifting to the service provider the burden of managing these apps (such as installing and upgrading the software, security, backups and disaster recovery).

### Software Development Kit (SDK) **Software Development Kits (SDKs)**

buy a software package from a software vendor—often a CD, DVD or web download. You then install that software on your computer and run it as needed. As new versions appear, you upgrade your software, often at considerable cost in time and money. This process can become cumbersome for organizations that must maintain tens of thousands of systems on a diverse array of computer equipment. With **Software as a Service (SaaS)**, the software runs on servers elsewhere on the Internet. When that server is updated, all clients worldwide see the new capabilities—no local installation is needed. You access the service through a browser. Browsers are quite portable, so you can run the same applications on a wide variety of computers from anywhere in the world. Salesforce.com, Google, and Microsoft’s Office Live and Windows Live all offer SaaS. **Platform as a Service (PaaS)** provides a computing platform for developing and running applications as a service over the web, rather than installing the tools on your computer. Some PaaS providers are Google App Engine, Amazon EC2 and Windows Azure™.

include the tools and documentation developers use to program applications.

**Fig. 1.33** | Software technologies. (Part 2 of 2.)

Software is complex. Large, real-world software applications can take many months or even years to design and implement. When large software products are under development, they typically are made available to the user communities as a series of releases, each more complete and polished than the last (Fig. 1.34).

Alpha *Alpha* software is the earliest release of a software product that’s still under active development. Alpha versions are often buggy, incomplete and unstable and are released to a relatively small number of

developers for testing new features, getting early feedback, etc.

**Fig. 1.34 | Software product-release terminology. (Part 1 of 2.)**  
66 Chapter 1 Introduction to Computers, the Internet and the Web

Beta *Beta* versions are released to a larger number of developers later in the development process after most major bugs have been fixed and new features are nearly complete. Beta software is more stable, but still subject to change.

Release candidates environment—the software is used on different systems, with varying constraints and for a variety of purposes. *Release candidates* are generally *feature complete*, (mostly) bug free and ready for use by the community, which provides a diverse testing

Final release Any bugs that appear in the release candidate are corrected, and eventually the final product is released to the general public. Software companies often distribute incremental updates over the Internet.

Continuous beta numbers. It's hosted in the *cloud* (not installed on your computer) and is Software that's developed using this approach (for example, Google search or Gmail) generally does not have version constantly evolving so that users always have the latest version.

**Fig. 1.34 | Software product-release terminology. (Part 2 of 2.)**

## 1.14 Keeping Up-to-Date with Information Technologies

Figure 1.35 lists key technical and business publications that will help you stay up-to-date with the latest news and trends and technology. You can also find a growing list of Internet- and web-related Resource Centers at [www.deitel.com/ResourceCenters.html](http://www.deitel.com/ResourceCenters.html).

AllThingsD [allthingsd.com](http://allthingsd.com)

Bloomberg BusinessWeek [www.businessweek.com](http://www.businessweek.com)

CNET [news.cnet.com](http://news.cnet.com)

Communications of the ACM [cacm.acm.org](http://cacm.acm.org)

Computerworld [www.computerworld.com](http://www.computerworld.com)

Engadget [www.engadget.com](http://www.engadget.com)

eWeek [www.eweek.com](http://www.eweek.com)

Fast Company [www.fastcompany.com](http://www.fastcompany.com)

Fortune [money.cnn.com/magazines/fortune](http://money.cnn.com/magazines/fortune)

GigaOM [gigaom.com](http://gigaom.com)

Hacker News [news.ycombinator.com](http://news.ycombinator.com)

IEEE Computer Magazine [www.computer.org/portal/web/computingnow/computer](http://www.computer.org/portal/web/computingnow/computer) InfoWorld  
[www.infoworld.com](http://www.infoworld.com)

Mashable [mashable.com](http://mashable.com)

PCWorld [www.pcworld.com](http://www.pcworld.com)

**Fig. 1.35 | Technical and business publications. (Part 1 of 2.)****1.14 Keeping Up-to-Date with Information Technologies 67**Slashdot [slashdot.org](http://slashdot.org)Stack Overflow [stackoverflow.com](http://stackoverflow.com)Technology Review [technologyreview.com](http://technologyreview.com)Techcrunch [techcrunch.com](http://techcrunch.com)The Next Web [thenextweb.com](http://thenextweb.com)The Verge [www.theverge.com](http://www.theverge.com)Wired [www.wired.com](http://www.wired.com)**Fig. 1.35 | Technical and business publications. (Part 2 of 2.)****Self-Review Exercises****1.1** Fill in the blanks in each of the following statements:

- a) Computers process data under the control of sets of instructions called . b) The key logical units of the computer are the , , , , and .
- c) The three types of languages discussed in the chapter are , and .
- d) The programs that translate high-level-language programs into machine language are called .
- e) is an operating system for mobile devices based on the Linux kernel and Java. f) software is generally feature complete, (supposedly) bug free and ready for use by the community.
- g) The Wii Remote, as well as many smartphones, use a(n) which allows the device to respond to motion.
- h) C is widely known as the development language of the operating system. i) is the new programming language for developing iOS and Mac apps.

**1.2** Fill in the blanks in each of the following sentences about the C environment. a) C

- programs are normally typed into a computer using a(n) program. b) In a C system, a(n) program automatically executes before the translation phase begins.
- c) The two most common kinds of preprocessor directives are and . d) The program combines the output of the compiler with various library functions to produce an executable image.
- e) The program transfers the executable image from disk to memory.

**1.3** Fill in the blanks in each of the following statements (based on Section 1.8): a)

- Objects have the property of —although objects may know how to communicate with one another across well-defined interfaces, they normally are not allowed to know how other objects are implemented.
- b) In object-oriented programming languages, we create to house the set of methods that perform tasks.
- c) With , new classes of objects are derived by absorbing characteristics of existing classes, then adding unique characteristics of their own.
- d) The size, shape, color and weight of an object are considered of the object's class.

## Answers to Self-Review Exercises

**1.1** a) programs. b) input unit, output unit, memory unit, central processing unit, arithmetic and logic unit, secondary storage unit. c) machine languages, assembly languages, high-level languages. d) compilers. e) Android. f) Release candidate. g) accelerometer. h) UNIX. i) Swift.

**1.2** a) editor. b) preprocessor. c) including other files in the file to be compiled, performing various text replacements. d) linker. e) loader.

**1.3** a) information hiding. b) classes. c) inheritance. d) attributes.

## Exercises

**1.4** Categorize each of the following items as either hardware or software:

- a) a microprocessor
- b) RAM
- c) Microsoft Visual Studio
- d) a preprocessor
- e) a scanner
- f) an internet browser

**1.5** Fill in the blanks in each of the following statements:

- a) Translator programs convert programs written in high-level languages into machine language.
- b) A multi-core processor implements multiple on a single integrated-circuit chip.
- c) A places a program in memory so that it can be executed.
- d) Programs in generally consist of strings of numbers that instruct computers to perform their most elementary operations one at a time.
- e) A is the smallest data item in a computer.
- f) are composed of characters or bytes.
- g) A is a collection of data organized for easy access and manipulation.
- h) C programs typically go through six phases to be executed. These are , , , , and .
- i) usually allow including other files and various text replacements.
- j) and are essentially reusable software components.

**1.6** Fill in the blanks in each of the following statements:

- a) allows software, platforms and infrastructure to be hosted on demand over the internet.
- b) , a Web 2.0 technology, helps Internet-based applications perform like desktop applications.
- c) , based on the Linux kernel and Java, is the fastest growing mobile and smart phone operating system.

**1.7** Discuss the meaning of each of the following phases of execution:

- a) linking
- b) loading
- c) execution

**1.8** What are standard input, output, and error streams?

Making a Difference 69

**1.9 (Internet Negatives)** Besides their numerous benefits, the Internet and the web have several downsides, such as privacy issues, identity theft, spam and malware. Research some of the negative aspects of the Internet. List five problems and describe what could possibly be done to help solve each.

**1.10 (Watch as an Object)** You are probably wearing on your wrist one of the most common types of objects—a watch. Discuss how each of the following terms and

concepts applies to the notion of a watch: object, attributes, behaviors, class, inheritance (consider, for example, an alarm clock), messages, encapsulation and information hiding.

## Making a Difference

Throughout the book we've included Making a Difference exercises in which you'll be asked to work on problems that really matter to individuals, communities, countries and the world.

**1.11 (Test-Drive: Carbon Footprint Calculator)** Some scientists believe that carbon emissions, especially from the burning of fossil fuels, contribute significantly to global warming and that this can be combatted if individuals take steps to limit their use of carbon-based fuels. Organizations and individuals are increasingly concerned about their "carbon footprints." Websites such as TerraPass

<http://www.terrapass.com/carbon-footprint-calculator-2/>

and Carbon Footprint

<http://www.carbonfootprint.com/calculator.aspx>

provide carbon-footprint calculators. Test-drive these calculators to determine your carbon foot print. Exercises in later chapters will ask you to program your own carbon-footprint calculator. To prepare for this, use the web to research the formulas for calculating carbon footprints.

**1.12 (Test-Drive: Body Mass Index Calculator)** Obesity causes significant increases in illnesses such as diabetes and heart disease. To determine whether a person is overweight or obese, you can use a measure called the body mass index (BMI). The United States Department of Health and Human Services provides a BMI calculator at <http://www.nhlbi.nih.gov/guidelines/obesity/BMI/bmicalc.htm>. Use it to calculate your own BMI. An exercise in Exercise 2.32 will ask you to program your own BMI calculator. To prepare for this, use the web to research the formulas for calculating BMI.

**1.13 (Attributes of Hybrid Vehicles)** In this chapter you learned some basics of classes. Now you'll "flesh out" aspects of a class called "Hybrid Vehicle." Hybrid vehicles are becoming increasingly popular, because they often get much better mileage than purely gasoline-powered vehicles. Browse the web and study the features of four or five of today's popular hybrid cars, then list as many of their hybrid-related attributes as you can. Some common attributes include city-miles-per-gallon and highway-miles-per-gallon. Also list the attributes of the batteries (type, weight, etc.).

**1.14 (Gender Neutrality)** Many people want to eliminate sexism in all forms of communication. You've been asked to create a program that can process a paragraph of text and replace gender-specific words with gender-neutral ones. Assuming that you've been given a list of gender-specific words and their gender-neutral replacements (e.g., replace "wife" with "spouse," "man" with "person," "daughter" with "child" and so on), explain the procedure you'd use to read through a paragraph of text and manually perform these replacements. How might your procedure generate a strange term like "woperchild?" In Chapter 4, you'll learn that a more formal term for "procedure" is "algorithm," and that an algorithm specifies the steps to be performed and the order in which to perform them.

## 70 Chapter 1 Introduction to Computers, the Internet and the Web

**1.15 (Privacy)** Some online e-mail services save all e-mail correspondence for some period of time. Suppose a disgruntled employee were to post all of the e-mail correspondences for millions of people, including yours, on the Internet. Discuss the issues.

**1.16 (Programmer Responsibility and Liability)** As a programmer in industry, you

may develop software that could affect people's health or even their lives. Suppose a software bug in one of your programs causes a cancer patient to receive an excessive dose during radiation therapy and that the person is severely injured or dies. Discuss the issues.

**1.17 (2010 “Flash Crash”)** An example of the consequences of our excessive dependence on computers was the so-called “flash crash” which occurred on May 6, 2010, when the U.S. stock market fell precipitously in a matter of minutes, wiping out trillions of dollars of investments, and then recovered within minutes. Research online the causes of this crash and discuss the issues it raises.

# Introduction to C

## Programming

### Objectives

In this chapter, you'll:

- Write simple C programs.
- Use simple input and output statements.
- Use the fundamental data types.
- Learn computer memory concepts.
- Use arithmetic operators.
- Learn the precedence of arithmetic operators.
- Write simple decision making statements.
- Begin focusing on secure C programming practices.

### 72 Chapter 2 Introduction to C Programming

#### 2.1 Introduction

#### 2.2 A Simple C Program: Printing a Line of Text

#### 2.3 Another Simple C Program:

#### Adding Two Integers

#### 2.4 Memory Concepts

#### 2.5 Arithmetic in C

#### 2.6 Decision Making: Equality and Relational Operators

#### 2.7 Secure C Programming

## 2.1 Introduction

The C language facilitates a structured and disciplined approach to computer-program design. In this chapter we introduce C programming and present several examples that illustrate many important features of C. Each example is analyzed one statement at a time. In Chapters 3 and 4 we present an introduction to structured programming in C. We then use the structured approach throughout the remainder of the C portion of the text. We provide the first of many “Secure C Programming” sections.

## 2.2 A Simple C Program: Printing a Line of Text

C uses some notations that may appear strange to people who have not programmed computers. We begin by considering a simple C program. Our first example prints a line of text. The program and its screen output are shown in Fig. 2.1.

```
1 // Fig. 2.1: fig02_01.c
2 // A first program in C.
3 #include <stdio.h>
4
5 // function main begins program execution
6 int main( void )
7 {
8     printf( "Welcome to C!\n" );
9 } // end function main
```

Welcome to C!

**Fig. 2.1** | A first program in C.

### Comments

Even though this program is simple, it illustrates several important features of the C language. Lines 1 and 2

```
// Fig. 2.1: fig02_01.c
// A first program in C
```

begin with `//`, indicating that these two lines are **comments**. You insert comments to **document programs** and improve program readability. Comments do *not* cause the computer to perform any action when the program is run—they’re *ignored* by the C compiler and

### 2.2 A Simple C Program: Printing a Line of Text 73

do *not* cause any machine-language object code to be generated. The preceding comment simply describes the figure number, filename and purpose of the program. Comments also help other people read and understand your program.

You can also use `/*...*/` **multi-line comments** in which everything from `/*` on the first line to `*/` at the end of the last line is a comment. We prefer `//`

comments because they're shorter and they eliminate common programming errors that occur with `/*...*/` comments, especially when the closing `*/` is omitted.

### *#include* **Preprocessor Directive**

Line 3

```
#include <stdio.h>
```

is a directive to the **C preprocessor**. Lines beginning with `#` are processed by the preprocessor *before* compilation. Line 3 tells the preprocessor to include the contents of the **standard input/output header** (`<stdio.h>`) in the program. This header contains information used by the compiler when compiling calls to standard input/output library functions such as `printf` (line 8). We explain the contents of headers in more detail in Chapter 5.

### **Blank Lines and White Space**

Line 4 is simply a blank line. You use blank lines, space characters and tab characters (i.e., “tabs”) to make programs easier to read. Together, these characters are known as **white space**. White-space characters are normally ignored by the compiler.

### **The *main* Function**

Line 6

```
int main( void )
```

is a part of every C program. The parentheses after `main` indicate that `main` is a program building block called a **function**. C programs contain one or more functions, one of which *must* be `main`. Every program in C begins executing at the function `main`. Functions can *return* information. The keyword `int` to the left of `main` indicates that `main` “returns” an integer (whole-number) value. We'll explain what it means for a function to “return a value” when we demonstrate how to create your own functions in Chapter 5. For now, simply include the keyword `int` to the left of `main` in each of your programs.

Functions also can *receive* information when they're called upon to execute. The `void` in parentheses here means that `main` does *not* receive any information. In Chapter 14, we'll show an example of `main` receiving information.

## **Good Programming Practice 2.1**

*Every function should be preceded by a comment describing the function's purpose.*

A left brace, `{`, begins the **body** of every function (line 7). A corresponding **right brace** ends each function (line 9). This pair of braces and the portion of the program between the braces is called a *block*. The block is an important program unit in C.

### **An Output Statement**

Line 8

```
printf( "Welcome to C!\n" );
```

74 Chapter 2 Introduction to C Programming

instructs the computer to perform an **action**, namely to print on the screen the **string** of characters marked by the quotation marks. A string is sometimes

called a **character string**, a **message** or a **literal**. The entire line, including the `printf` function (the “f” stands for “formatted”), its **argument** within the parentheses and the semicolon (;), is called a **statement**. Every statement must end with a semicolon (also known as the **statement terminator**). When the preceding `printf` statement is executed, it prints the message `Welcome to C!` on the screen. The characters normally print exactly as they appear between the double quotes in the `printf` statement.

## Escape Sequences

Notice that the characters `\n` were not printed on the screen. The backslash (`\`) as used here is called an **escape character**. It indicates that `printf` is supposed to do something out of the ordinary. When encountering a backslash in a string, the compiler looks ahead at the next character and combines it with the backslash to form an **escape sequence**. The escape sequence `\n` means **newline**. When a newline appears in the string output by a `printf`, the newline causes the cursor to position to the beginning of the next line on the screen. Some common escape sequences are listed in Fig. 2.2.

- `\n` Newline. Position the cursor at the beginning of the next line. `\t` Horizontal tab. Move the cursor to the next tab stop. `\a` Alert. Produces a sound or visible alert without changing the current cursor position.
- `\\` Backslash. Insert a backslash character in a string.
- `\"` Double quote. Insert a double-quote character in a string. **Fig. 2.2** |

### Some common escape sequences .

Because the backslash has special meaning in a string, i.e., the compiler recognizes it as an escape character, we use a *double* backslash (`\\`) to place a single backslash in a string. Printing a double quote also presents a problem because double quotes mark the boundaries of a string—such quotes are not printed. By using the escape sequence `\"` in a string to be output by `printf`, we indicate that `printf` should display a double quote. The right brace, `}`, (line 9) indicates that the end of `main` has been reached.

## Good Programming Practice 2.2

*Add a comment to the line containing the right brace, `}`, that closes every function, including `main`.*

We said that `printf` causes the computer to perform an **action**. As any program executes, it performs a variety of actions and makes **decisions**. Section 2.6 discusses decision making. Chapter 3 discusses this **action/decision model** of programming in depth.

## The Linker and Executables

Standard library functions like `printf` and `scanf` are *not* part of the C programming language. For example, the compiler *cannot* find a spelling error in `printf` or `scanf`. When

the compiler compiles a `printf` statement, it merely provides space in the object program for a “call” to the library function. But the compiler does *not* know where the library functions are—the *linker* does. When the linker runs, it locates the library functions and inserts the proper calls to these library functions in the object program. Now the object program is complete and ready to be executed. For this reason, the linked program is called an **executable**. If the function name is misspelled, the *linker* will spot the error, because it will not be able to match the name in the C program with the name of any known function in the libraries.

## Common Programming Error 2.1

*Mistyping the name of the output function `printf` as `print` in a program.*

## Good Programming Practice 2.3

*Indent the entire body of each function one level of indentation (we recommend three spaces) within the braces that define the body of the function. This indentation emphasizes the functional structure of programs and helps make them easier to read.*

## Good Programming Practice 2.4

*Set a convention for the indent size you prefer and then uniformly apply that convention. The tab key may be used to create indents, but tab stops can vary. Professional style guides often recommend using spaces rather than tabs.*

### Using Multiple `printf`s

The `printf` function can print `Welcome to C!` several different ways. For example, the program of Fig. 2.3 produces the same output as the program of Fig. 2.1. This works because each `printf` resumes printing where the previous `printf` stopped printing. The first `printf` (line 8) prints `Welcome` followed by a space (but no newline), and the second `printf` (line 9) begins printing on the *same* line immediately following the space.

```
1 // Fig. 2.3: fig02_03.c
2 // Printing on one line with two printf statements.
3 #include <stdio.h>
4
5 // function main begins program execution
6 int main( void )
7 {
8     printf(
9         "Welcome to C!
10 } // end function main
```

```
Welcome to C!
```

**Fig. 2.3** | Printing one line with two `printf` statements.

One `printf` can print *several* lines by using additional newline characters as in Fig. 2.4. Each time the `\n` (newline) escape sequence is encountered, output continues at the beginning of the next line.

## 76 Chapter 2 Introduction to C Programming

```
1 // Fig. 2.4: fig02_04.c
2 // Printing multiple lines with a single printf. 3 #include <stdio.h>
4
5 // function main begins program execution 6 int main( void
7 {
8 printf( "Welcome to C!\n" );
```



```
9 } // end function main
```

```
Welcome
to
C!
```

**Fig. 2.4** | Printing multiple lines with a single `printf`.

## 2.3 Another Simple C Program: Adding Two Integers

Our next program uses the Standard Library function `scanf` to obtain two integers typed by a user at the keyboard, computes the sum of these values and prints the result using `printf`. The program and sample output are shown in Fig. 2.5. [In the input/output dialog of Fig. 2.5, we emphasize the numbers entered by the user in **bold**.]

```
1 // Fig. 2.5: fig02_05.c
2 // Addition program.
3 #include <stdio.h>
4
5 // function main begins program execution
6 int main( void )
7 {
8 9 10
11 printf( "Enter first integer\n" ); // prompt
12 13
14 printf( "Enter second integer\n" ); // prompt
15 16
17
```

```
int integer1; // first r
```

```
int integer2; // second
```

```
scanf( "%d", &in
```

```
scanf( "%d", &integer2 ); // read an integer
```

```

17     int sum; // variable
18 sum = integer1 + integer2; // assign total to sum
19
20
21 } // end function main

```

```

Enter first integer
45
Enter second integer
72
Sum is 117

```

**Fig. 2.5** | Addition program.

## 2.3 Another Simple C Program: Adding Two Integers 77

The comment in lines 1–2 states the purpose of the program. As we stated earlier, every program begins execution with `main`. The left brace `{` (line 7) marks the beginning of the body of `main`, and the corresponding right brace `}` (line 21) marks the end of `main`.

### **Variables and Variable Definitions**

Lines 8–9

```

8     int integer1; // first number to be entered by user
9     int integer2; // second number to be entered by user

```

are **definitions**. The names `integer1` and `integer2` are the names of **variables**—locations in memory where values can be stored for use by a program. These definitions specify that variables `integer1` and `integer2` are of type `int`, which means that they'll hold *integer* values, i.e., whole numbers such as 7, −11, 0, 31914 and the like.

### **Define Variables Before They Are Used**

All variables must be defined with a name and a data type *before* they can be used in a program. The C standard allows you to place each variable definition *anywhere* in `main` before that variable's first use in the code (though some older compilers do not allow this). You'll see later why you should define variables *close* to their first use.

### **Defining Multiple Variables of the Same Type in One Statement**

The preceding definitions could be combined into a single definition as follows:

```

10    int integer1, integer2;

```

but that would have made it difficult to associate comments with each of the variables, as we did in lines 8–9.

### **Identifiers and Case Sensitivity**

A variable name in C can be any valid **identifier**. An identifier is a series of

characters consisting of letters, digits and underscores (\_) that does *not* begin with a digit. C is **case sensitive**—uppercase and lowercase letters are *different* in C, so `a1` and `A1` are *different* identifiers.

## Common Programming Error 2.2

*Using a capital letter where a lowercase letter should be used (for example, typing `Main` instead of `main`).*

## Error-Prevention Tip 2.1

*Avoid starting identifiers with the underscore character (\_) to prevent conflicts with compiler-generated identifiers and standard library identifiers.*

## Good Programming Practice 2.5

*Choosing meaningful variable names helps make a program self-documenting—that is, fewer comments are needed.*

## Good Programming Practice 2.6

*The first letter of an identifier used as a simple variable name should be a lowercase letter. Later in the text we'll assign special significance to identifiers that begin with a capital letter and to identifiers that use all capital letters.*

## 78 Chapter 2 Introduction to C Programming

## Good Programming Practice 2.7

*Multiple-word variable names can help make a program more readable. Separate the words with underscores as in `total_commissions`, or, if you run the words together, begin each word after the first with a capital letter as in `totalCommissions`. The latter style—often called camel casing because the pattern of uppercase and lowercase letters resembles the silhouette of a camel—is preferred.*

## Prompting Messages

Line 11

```
printf( "Enter first integer\n" ); // prompt
```

displays the literal "Enter first integer" and positions the cursor to the beginning of the next line. This message is called a **prompt** because it tells the user to take a specific action.

## The `scanf` Function and Formatted Inputs

Line 12

```
scanf( "%d", &integer1 ); // read an integer
```

uses **`scanf`** (the "r" stands for "formatted") to obtain a value from the user. The function reads from the *standard input*, which is usually the keyboard.

This `scanf` has two arguments, `"%d"` and `&integer1`. The first, the **format control string**, indicates the *type* of data that should be entered by the user. The **`%d` conversion specifier** indicates that the data should be an integer (the letter `d` stands for "decimal integer"). The `%` in this context is treated by `scanf` (and `printf` as we'll see) as a special character that begins a conversion specifier.

The second argument of `scanf` begins with an ampersand (&)—called the **address operator**—followed by the variable name. The &, when combined with the variable name, tells `scanf` the location (or address) in memory at which the variable `integer1` is stored. The computer then stores the value that the user enters for `integer1` at that location. The use of ampersand (&) is often confusing to novice programmers or to people who have programmed in other languages that do not require this notation. For now, just remember to precede each variable in every call to `scanf` with an ampersand. Some exceptions to this rule are discussed in Chapters 6 and 7. The use of the ampersand will become clear after we study *pointers* in Chapter 7.

## Good Programming Practice 2.8

*Place a space after each comma (,) to make programs more readable.*

When the computer executes the preceding `scanf`, it waits for the user to enter a value for variable `integer1`. The user responds by typing an integer, then pressing the **Enter key** (sometimes labeled as the *Return* key) to send the number to the computer. The computer then assigns this number, or value, to the variable `integer1`. Any subsequent references to `integer1` in this program will use this same value. Functions `printf` and `scanf` facilitate interaction between the user and the computer. This interaction resembles a dialogue and is often called **interactive computing**.

### 2.3 Another Simple C Program: Adding Two Integers 79

#### **Prompting for and Inputting the Second Integer**

Line 14

```
printf( "Enter second integer\n" ); // prompt
```

displays the message `Enter second integer` on the screen, then positions the cursor to the beginning of the next line. This `printf` also prompts the user to take action. Line 15

```
scanf( "%d", &integer2 ); // read an integer
```

obtains a value for variable `integer2` from the user.

#### **Defining the `sum` Variable**

Line 17

```
int sum; // variable in which sum will be stored
```

defines the variable `sum` of type `int` just before its first use in line 18.

#### **Assignment Statement**

The **assignment statement** in line 18

```
sum = integer1 + integer2; // assign total to sum
```

calculates the total of variables `integer1` and `integer2` and assigns the result to variable `sum` using the **assignment operator** `=`. The statement is read as, “`sum` gets the value of the expression `integer1 + integer2`.” Most calculations are performed in assignments. The `=` operator and the `+` operator are called **binary operators** because each has *two operands*. The `+` operator’s operands are `integer1` and `integer2`. The `=` operator’s two operands are `sum` and the value of the expression `integer1 + integer2`.

## Good Programming Practice 2.9

*Place spaces on either side of a binary operator. This makes the operator stand out and makes the program more readable.*

## Common Programming Error 2.3

*A calculation in an assignment statement must be on the right side of the = operator. It's a compilation error to place a calculation on the left side of an assignment operator.*

### ***Printing with a Format Control String***

Line 20

```
printf( "Sum is %d\n", sum ); // print sum
```

calls function `printf` to print the literal `Sum is` followed by the numerical value of variable `sum` on the screen. This `printf` has two arguments, `"Sum is %d\n"` and `sum`. The first is the format control string. It contains some literal characters to be displayed and the conversion specifier `%d` indicating that an integer will be printed. The second argument specifies the value to be printed. The conversion specifier for an integer is the same in both `printf` and `scanf`—this is true for most C data types.

### ***Combining a Variable Definition and Assignment Statement***

You can assign a value to a variable in its definition—this is known as **initializing** the variable. For example, lines 17–18 can be combined into the statement